

PROJECT HELIX CLUSTER OS — COMPLETE ARCHITECTURE BLUEPRINT

Version 1.0 | Date: 2026-05-30

TABLE OF CONTENTS

1. Executive Summary
2. Architectural Principles
3. System Overview
4. Layered Architecture
5. Core Subsystems
6. Microservices Specification
7. Network Architecture
8. Data Architecture
9. Security Architecture
10. Execution Modes
11. Component Interaction Flows
12. Database Schemas
13. API Specifications
14. Implementation Phases
15. Risk Analysis & Mitigation

1. EXECUTIVE SUMMARY

Vision

Helix Cluster OS is a distributed computing abstraction layer that binds heterogeneous computers (Intel i7, AMD Ryzen 9, Apple Silicon M3 Pro, with assorted GPUs) into a single coherent compute block. All CPUs, GPUs, RAM, storage, and network resources appear as one unified pool. A user starts a session (like tmux) and work transparently utilizes all available resources across the cluster. Nodes dynamically join, leave, go offline, and come back — all fully automatically.

Key Differentiators

- **No kernel modifications** — Pure user-space implementation
- **Heterogeneous hardware support** — Intel, AMD, Apple CPUs; NVIDIA, AMD, Intel, Apple GPUs
- **Transparent session distribution** — tmux-like UX extended across cluster
- **LLM-powered optimization** — Self-tuning “brain” that learns and improves
- **Fully automated setup** — Single-command install, zero configuration
- **Two execution modes** — Batch (AOSP builds) and Interactive (AI agents)
- **Production-proven foundations** — Built on Kubernetes patterns, etcd, WireGuard

Target Hardware

Category	Supported Hardware
CPUs	Intel Core i7/i9, AMD Ryzen 9, Apple Silicon M3/M4 Pro/Max
GPUs	NVIDIA RTX/GTX/A100/H100, AMD Radeon/Instinct, Intel Arc/Xe, Apple Metal
Network	Gigabit Ethernet (minimum), Wi-Fi, SSH tunnel, VPN mesh
OS	Linux (primary), macOS (Apple Silicon), Windows (WSL2)

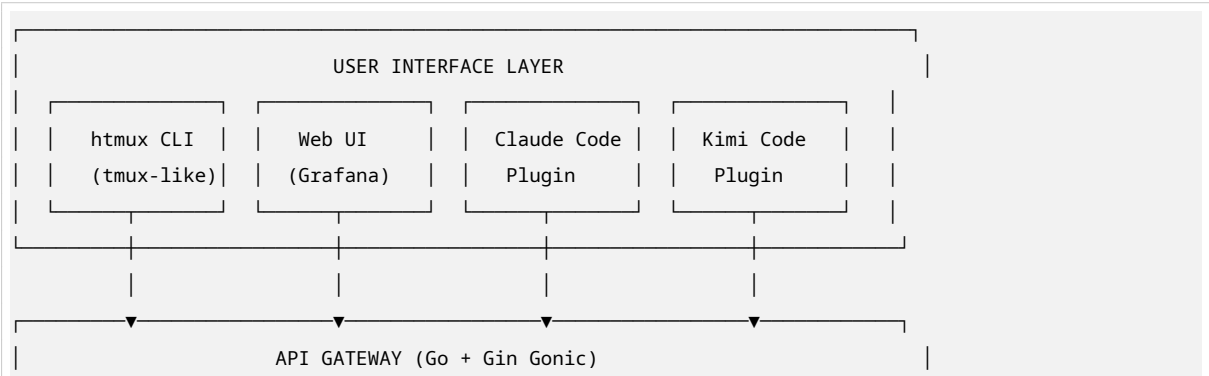
2. ARCHITECTURAL PRINCIPLES

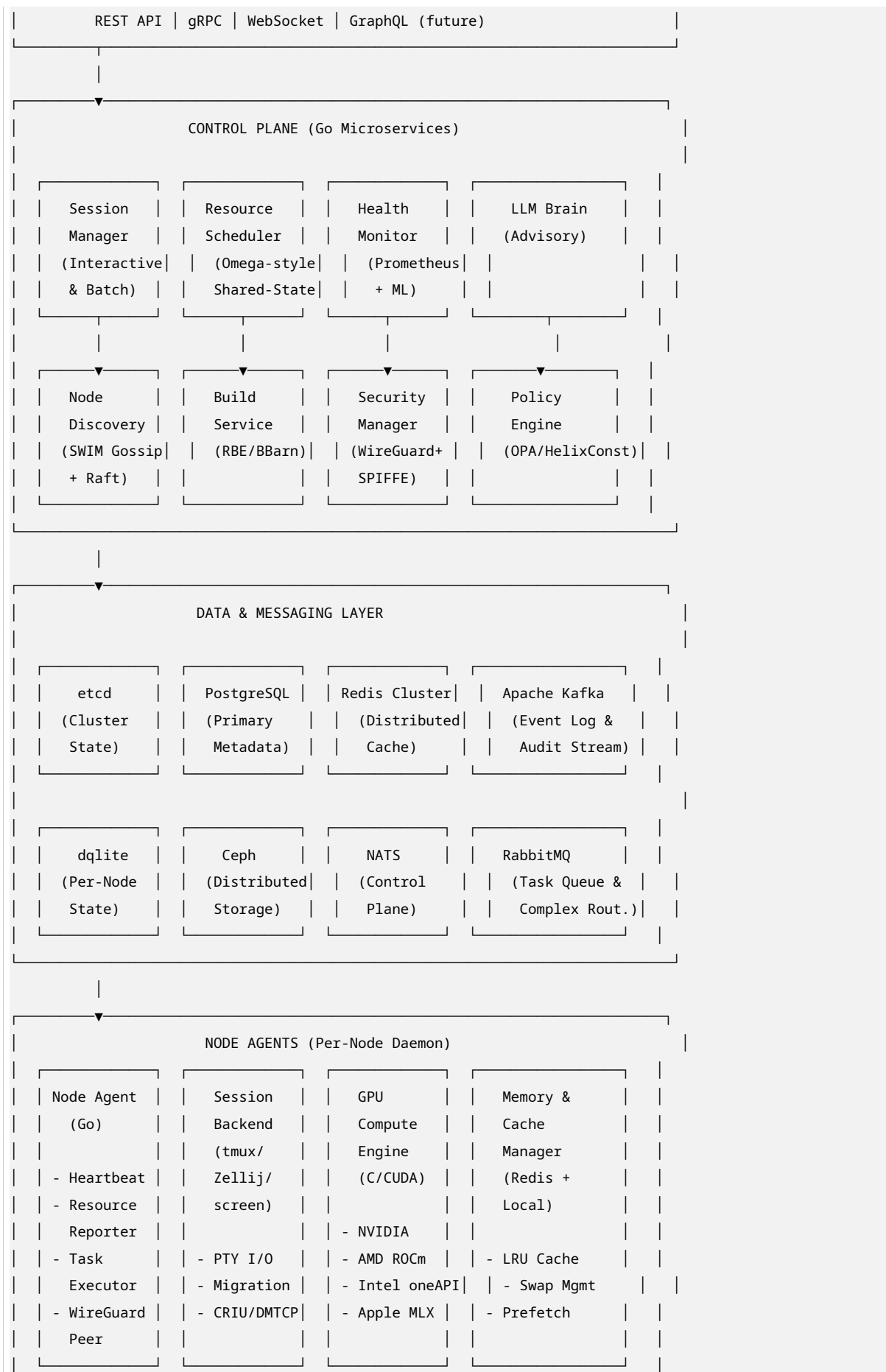
Derived from cross-dimension research insights:

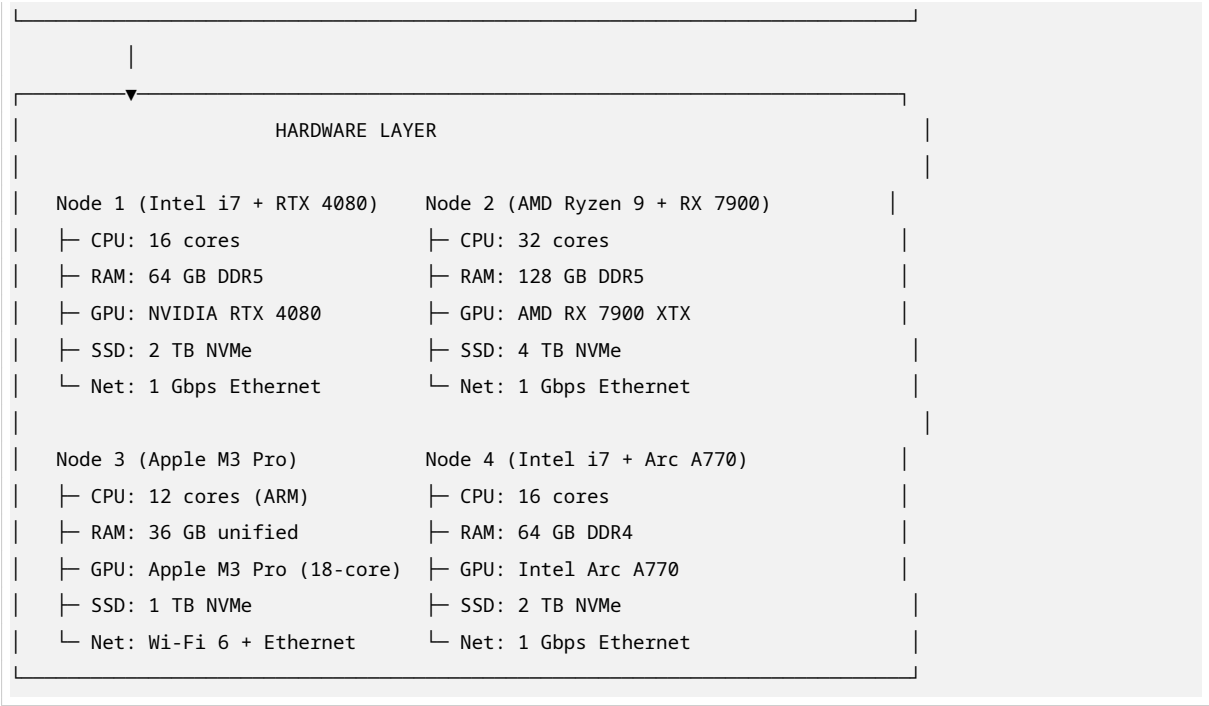
#	Principle	Source Insight
1	Resource Disaggregation + Proven Orchestration	LegoOS splitkernel concepts + Kubernetes Omega scheduling
2	Session-First UX	Terminal multiplexing as the primary user abstraction
3	Capability Negotiation	HTCondor ClassAds pattern for all heterogeneous resources
4	Pessimistic Local, Optimistic Global	Local ACID + Saga pattern for distributed consistency
5	Advisory LLM, Binding Policy	LLM suggests, policy engine decides
6	Graceful Degradation	Lose capacity, never correctness
7	Zero-Copy Data Paths	Software optimization over hardware upgrades
8	Invisible Security	WireGuard + SPIFFE automatic, no user configuration
9	Safety-Critical Testing	TLA+ formal verification + chaos engineering
10	Mode-Driven Architecture	Separate Batch and Interactive execution paths
11	Zig + Go + C Stack	Zig for systems, Go for services, C for kernel/GPU
12	Flawless Setup	<10 minutes, zero config, fully automated

3. SYSTEM OVERVIEW

High-Level Architecture Diagram







4. LAYERED ARCHITECTURE

Layer Stack

Layer	Technologies	Purpose
L7: User Interface	htmux CLI, Web UI (React), IDE Plugins	Human interaction
L6: API Gateway	Go + Gin Gonic, gRPC-Gateway	Unified API surface
L5: Control Plane	Go microservices	Cluster management logic
L4: Data & Messaging	etcd, PostgreSQL, Redis, Kafka, NATS	State, messages, events
L3: Node Runtime	Go agents, Zig libraries, C extensions	Per-node execution
L2: System Primitives	Zig (network, serialization), C (GPU, kernel)	High-performance primitives
L1: Hardware Abstraction	DRA/CDI, HAMi, SPIFFE, WireGuard	Hardware interfaces
L0: Physical Hardware	CPU, GPU, RAM, SSD, NIC	Physical resources

Technology Stack Matrix

Component	Primary	Secondary	Rationale
Microservices	Go + Gin	Rust (future)	Proven for distributed systems
System Layer	Zig	C, Odin	Memory safety, C ABI compat
GPU Compute	C/CUDA	HIP, SYCL, Metal	Vendor-native performance

Table 4 – continued

Component	Primary	Secondary	Rationale
CLI/Tools	Go	BASH	Fast compilation, cross-platform
Setup Wizards	BASH + Go	—	Ubiquitous, zero dependencies
Message Bus (Control)	NATS + JetStream	RabbitMQ	Lightweight, fast, durable
Event Streaming	Apache Kafka 4.0	—	Audit logs, event sourcing
Database (Primary)	PostgreSQL 16+	—	ACID, mature, proven
Database (Local)	SQLite (dqlite)	rqlite	Embedded, Raft-replicated
Cache	Redis Cluster 7+	—	Sub-ms, distributed, HA
Consensus	etcd (Raft)	HashiCorp Raft	Kubernetes-proven
Storage	Ceph	NFS, Lustre	Distributed, self-healing
Observability	Prometheus + Grafana	OpenTelemetry	Industry standard
ML/Forecasting	Python (isolated)	Go ONNX runtime	LSTM, anomaly detection
LLM Integration	Go SDK (LLMsVerifier)	REST API	Mandatory verification
Mesh VPN	WireGuard	Headscale	~8 Gbps, <1ms latency
Serialization	Cap'n Proto	FlatBuffers, Protobuf	Zero-copy messaging
Data Transfer	Apache Arrow Flight	gRPC streaming	95% of RDMA bandwidth
Container Runtime	containerd	CRI-O	Industry standard

5. CORE SUBSYSTEMS

5.1 Node Discovery & Membership Service (NDMS)

Purpose: Handle dynamic node join/leave, cluster topology maintenance, and failure detection.

Architecture Pattern: SWIM gossip protocol + Raft consensus for cluster state

Key Components:

```
// Node Registry (etcd-backed)
type Node struct {
    ID            string           `json:"id"`           // UUID v4
    Hostname      string           `json:"hostname"`
    IPAddresses   []string         `json:"ip_addresses"` // All network interfaces
    WireGuardPubKey string         `json:"wg_pubkey"`
    SPIFFE_ID     string           `json:"spiffe_id"`
    Status        NodeStatus       `json:"status"`       // JOINING, ACTIVE, SUSPECT, LEFT, FAILED
    Role          NodeRole         `json:"role"`         // WORKER, CONTROL, HYBRID
    Resources     NodeResources    `json:"resources"`    // Fingerprinted capabilities
    Capabilities  []Capability     `json:"capabilities"` // Advertised services
    Labels        map[string]string `json:"labels"`       // User-defined tags
    JoinedAt      time.Time        `json:"joined_at"`
    LastSeen      time.Time        `json:"last_seen"`
    Version       string           `json:"version"`      // Cluster OS version
}
```

```

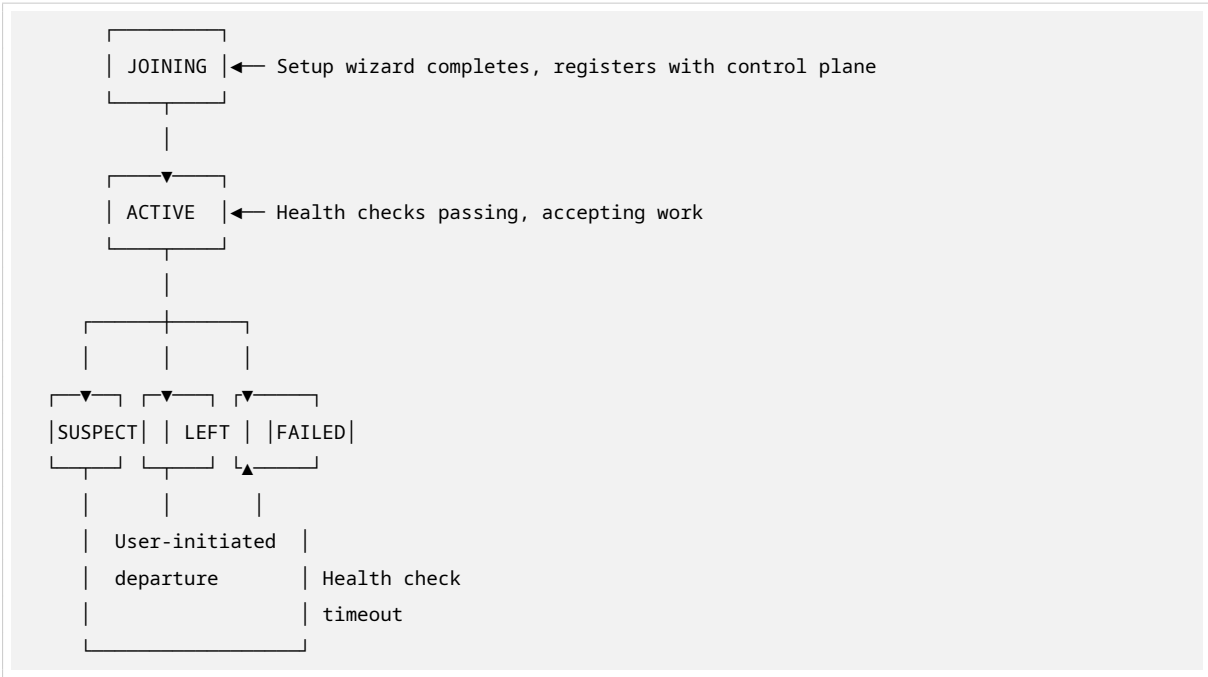
    Region      string      `json:"region"`      // Physical/zone location
}

type NodeResources struct {
    CPU          CPUResources    `json:"cpu"`
    Memory       MemoryResources `json:"memory"`
    GPUs         []GPUResource   `json:"gpus"`
    Storage      []StorageResource `json:"storage"`
    Network      NetworkResources `json:"network"`
}

type Capability struct {
    Name          string      `json:"name"`      // e.g., "cuda:12.4", "rocm:6.0"
    Type          CapabilityType `json:"type"`      // GPU, CPU_FEATURE, STORAGE, NETWORK
    Version       string      `json:"version"`
    Quantity      int         `json:"quantity"`
    Attributes    map[string]string `json:"attributes"` // Vendor-specific details
}

```

State Machine:



Algorithms:

- **SWIM Gossip:** Each node randomly selects K peers (default K=3) and sends ping every T seconds (default T=1s)
- **Phi Accrual Failure Detector:** $\phi(t) = 0.434 * t / \text{mean_interval}$. Node marked SUSPECT at $\phi > 5$, FAILED at $\phi > 8$
- **Raft Consensus:** Cluster membership changes use single-server joint consensus (Raft 3.4+ protocol)

API Endpoints:

POST	/v1/nodes/join	# New node registration
POST	/v1/nodes/{id}/heartbeat	# Health heartbeat
POST	/v1/nodes/{id}/leave	# Graceful departure
GET	/v1/nodes	# List all nodes
GET	/v1/nodes/{id}	# Node details
GET	/v1/nodes/{id}/resources	# Current resource availability
PUT	/v1/nodes/{id}/labels	# Update node labels
GET	/v1/topology	# Cluster topology graph
WS	/v1/nodes/watch	# WebSocket: real-time node events

5.2 Resource Aggregator & Scheduler (RAS)

Purpose: Aggregate resources from all nodes into unified pool and schedule workloads using Omega-model shared-state with optimistic concurrency.

Architecture Pattern: Omega shared-state + HTCondor ClassAds capability negotiation

Key Components:

```
// Shared State (etcd-backed with optimistic concurrency)
type ResourcePool struct {
    TotalResources      ResourceSnapshot `json:"total"` // Sum across all ACTIVE nodes
    AvailableResources  ResourceSnapshot `json:"available"` // After reservations
    ReservedResources   ResourceSnapshot `json:"reserved"` // Active reservations
    Utilization         UtilizationMetrics `json:"utilization"`
    UpdatedAt           time.Time        `json:"updated_at"`
    Revision            uint64           `json:"revision"` // For optimistic concurrency
}

type ResourceSnapshot struct {
    CPUShares    int64      `json:"cpu_shares"` // In millicores (1000 = 1 core)
    MemoryBytes  int64      `json:"memory_bytes"`
    GPUDevices   []GPUAllocation `json:"gpu_devices"`
    StorageBytes int64      `json:"storage_bytes"`
}

// ClassAds-style Capability Negotiation
type ResourceRequest struct {
    ID            string      `json:"id"`
    SessionID     string      `json:"session_id"`
    Priority      int         `json:"priority"` // 0-100, higher = more important
    Requirements  string      `json:"requirements"` // ClassAds expression
    Rank          string      `json:"rank"` // Preference expression
    Resources     ResourceSpec
    Duration      *time.Duration `json:"duration,omitempty"`
    Mode          ExecutionMode `json:"mode"` // BATCH or INTERACTIVE
}

// Example Requirements expression:
// "(TARGET.CPU_ARCH == 'x86_64' || TARGET.CPU_ARCH == 'arm64') &&
```

```
// TARGET.GPU.CUDA_VERSION >= '12.0' &&
// TARGET.MEMORY >= 8589934592 &&
// TARGET.LABELS['zone'] == 'home'"
```

Scheduler Pipeline (12 extension points, Kubernetes-style):

- 1. QueueSort — Priority ordering of pending requests
- 2. PreFilter — Quick reject (e.g., no nodes available)
- 3. Filter — Hard constraints (requirements matching)
- 4. PostFilter — Fallback/preemption check
- 5. PreScore — Prepare scoring data
- 6. Score — Preference ranking (rank expression)
- 7. Reserve — Pessimistic resource reservation
- 8. Permit — Async approval (LLM Brain can intervene)
- 9. PreBind — Prepare binding (network setup, volume mount)
- 10. Bind — Commit placement decision
- 11. PostBind — Notify, start metrics collection
- 12. Unreserve — Release reservation on failure

Scheduling Plugins:

Plugin	Extension Points	Purpose
NodeResourcesFit	Filter, Score	CPU/Memory/GPU matching
NodeAffinity	Filter, Score	Label-based node selection
TopologyAware	Filter, Score	NUMA-aware placement
CapabilityMatch	Filter	ClassAds requirements evaluation
PrioritySort	QueueSort	Priority-based ordering
GangScheduling	Filter, Permit	All-or-nothing for distributed jobs
LoadAware	Score	Prefer underutilized nodes
LocalityAware	Score	Data locality optimization

API Endpoints:

POST	/v1/schedule	# Submit resource request
GET	/v1/schedule/{id}/status	# Check scheduling status
DELETE	/v1/schedule/{id}	# Cancel request
GET	/v1/pool	# View resource pool
GET	/v1/pool/utilization	# Cluster utilization metrics
POST	/v1/reserve	# Reserve resources
POST	/v1/reserve/{id}/release	# Release reservation
GET	/v1/capabilities	# List all cluster capabilities
WS	/v1/pool/watch	# Real-time pool changes

5.3 Session Manager (SM)

Purpose: Provide distributed session management that extends tmux-like experience across the cluster. Support multiple backends (tmux, Zellij, screen).

Architecture Pattern: Abstract session backend + distributed I/O forwarding + CRIU migration

Key Components:

```
// Session abstraction (backend-agnostic)
type Session struct {
    ID          string          `json:"id"`           // UUID v4
    Name        string          `json:"name"`
    Owner       string          `json:"owner"`       // SPIFFE identity
    Status      SessionStatus `json:"status"`      // CREATING, RUNNING, MIGRATING, PAUSED, TERMINATED
    Mode        ExecutionMode `json:"mode"`        // INTERACTIVE or BATCH
    Backend     BackendType  `json:"backend"`     // TMUX, ZELLIJ, SCREEN, NATIVE
    BackendID   string          `json:"backend_id"`  // Backend-specific ID
    NodeID      string          `json:"node_id"`     // Current executing node
    Resources   ResourceAllocation `json:"resources"`
    Windows     []Window        `json:"windows"`
    CreatedAt   time.Time       `json:"created_at"`
    LastActive  time.Time       `json:"last_active"`
    MigrationHistory []MigrationRecord `json:"migration_history"`
    CRDTSyncState map[string]interface{} `json:"crdt_sync"` // CRDT state for distributed windows
}

type Window struct {
    ID          string          `json:"id"`
    Name        string          `json:"name"`
    Panes       []Pane          `json:"panes"`
    Layout      LayoutType      `json:"layout"`
    Active      bool            `json:"active"`
    CRDTState   WindowCRDT      `json:"crdt_state"` // Yjs-style CRDT for distributed sync
}

type Pane struct {
    ID          string          `json:"id"`
    Command     string          `json:"command"`     // Running command
    WorkingDir  string          `json:"working_dir"`
    Environment map[string]string `json:"env"`
    PTYSocket   string          `json:"pty_socket"`  // Unix socket for I/O
    Resources   ResourceAllocation `json:"resources"`
    NodeID      string          `json:"node_id"`     // May differ from session node (distributed pane)
}
```

Session Backend Interface:

```
type SessionBackend interface {
    // Lifecycle
    Create(config SessionConfig) (*Session, error)
    Attach(sessionID string, client Client) (PTYStream, error)
    Detach(sessionID string, client Client) error
}
```

```

    Terminate(sessionID string) error

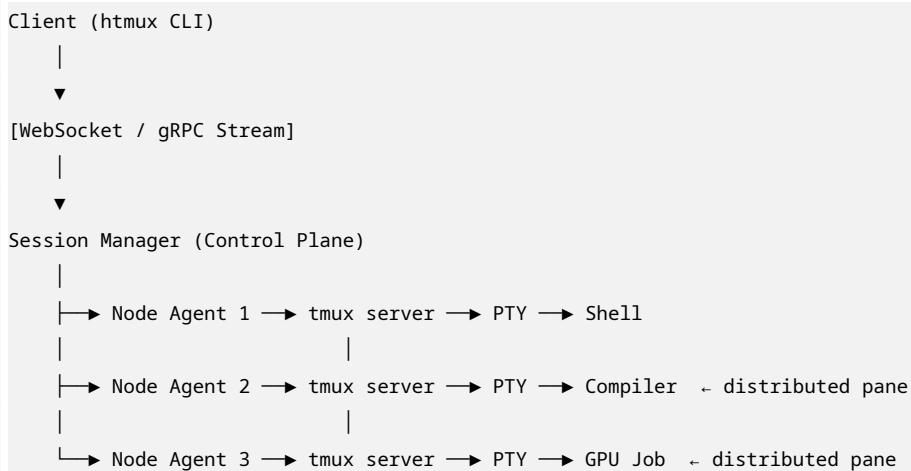
    // I/O
    SendInput(sessionID string, paneID string, data []byte) error
    Resize(sessionID string, paneID string, cols, rows int) error
    SubscribeOutput(sessionID string) (<-chan OutputEvent, error)

    // Migration
    Checkpoint(sessionID string) (*Checkpoint, error)
    Restore(checkpoint *Checkpoint, targetNode string) (*Session, error)

    // Queries
    ListSessions() ([]Session, error)
    GetSession(id string) (*Session, error)
    GetWindows(sessionID string) ([]Window, error)
    GetPanes(sessionID string, windowID string) ([]Pane, error)
}

```

Distributed I/O Forwarding Architecture:



Migration Flow (CRIU-based):

1. Scheduler decides to migrate session S from Node A to Node B
2. SM sends SIGSTOP to session S (freeze)
3. SM invokes CRIU checkpoint on Node A
 - Dumps process state (memory, registers, file descriptors)
 - TCP_REPAIR captures socket state
 - PTY state captured
4. Checkpoint data streamed to Node B via Arrow Flight
5. CRIU restore on Node B
 - Processes recreated with same PIDs
 - TCP connections reestablished (if IP same) or proxied
 - PTY reattached to new backend
6. SM updates routing table: S now on Node B
7. Client streams automatically redirected (EternalTerminal-style)
8. SIGCONT to resume session

API Endpoints:

POST	/v1/sessions	# Create session
POST	/v1/sessions/{id}/attach	# Attach to session (WebSocket upgrade)
POST	/v1/sessions/{id}/detach	# Detach (session continues)
POST	/v1/sessions/{id}/terminate	# Kill session
POST	/v1/sessions/{id}/migrate	# Migrate to different node
GET	/v1/sessions	# List sessions
GET	/v1/sessions/{id}	# Session details
GET	/v1/sessions/{id}/windows	# List windows
GET	/v1/sessions/{id}/windows/{wid}/panes	# List panes
POST	/v1/sessions/{id}/windows	# Create window
POST	/v1/sessions/{id}/windows/{wid}/panes	# Create pane (may be on different node!)
WS	/v1/sessions/{id}/stream	# I/O stream (bidirectional WebSocket)

5.4 GPU Compute Engine (GCE)

Purpose: Abstract NVIDIA, AMD, Intel, and Apple GPUs into a unified compute pool with capability-based scheduling.

Architecture Pattern: Kubernetes DRA + HAMI-style interception + SYCL cross-compilation

Key Components:

```
// GPU Resource Descriptor (DRA-compatible)
type GPUDevice struct {
    ID          string          `json:"id"`
    NodeID      string          `json:"node_id"`
    Vendor      GPUVendor        `json:"vendor"`      // NVIDIA, AMD, INTEL, APPLE
    Model       string          `json:"model"`       // RTX 4080, MI300X, etc.
    DriverVersion string        `json:"driver_version"`
    API         GPUAPI          `json:"api"`         // CUDA, ROCm, oneAPI, Metal
    APIVersion  string          `json:"api_version"`

    // Capabilities (ClassAds-style)
    TotalMemory   int64          `json:"total_memory"`
    AvailableMemory int64        `json:"available_memory"`
    ComputeUnits  int            `json:"compute_units"` // SMs, CUs, Xe-cores, etc.

    // Feature flags
    Features map[string]bool `json:"features"` // tensor_cores, ray_tracing, etc.

    // DRA Attributes
    Attributes map[string]string `json:"attributes"`

    Status GPUStatus      `json:"status"` // AVAILABLE, ALLOCATED, UNHEALTHY
}

// GPU Request (submitted to scheduler)
type GPURequest struct {
    Count    int            `json:"count"`
    Vendor   *GPUVendor    `json:"vendor,omitempty"` // "NVIDIA" or null for any
```

```
MinMemory    int64          `json:"min_memory"`           // Per GPU
API           *GPUAPI         `json:"api,omitempty"`         // "CUDA" or null for any
MinVersion    string        `json:"min_version,omitempty"` // e.g., "12.0"
Features      []string      `json:"features,omitempty"`   // Required features
Sharing       GPUSharingMode `json:"sharing"`             // EXCLUSIVE, MPS, TIME_SLICE
}
```

GPU Backend Interface:

```
type GPUBackend interface {
    // Discovery
    DetectDevices() ([]GPUDevice, error)
    GetDeviceStatus(id string) (*GPUDeviceStatus, error)

    // Execution
    Execute(ctx context.Context, spec ComputeSpec) (*ComputeResult, error)
    ExecuteDistributed(ctx context.Context, spec DistributedComputeSpec) (<-chan ComputeEvent, error)

    // Memory
    AllocateMemory(deviceID string, size int64) (*MemoryAllocation, error)
    FreeMemory(alloc *MemoryAllocation) error

    // Sharing
    EnableMPS(deviceID string, fraction float64) error
    DisableMPS(deviceID string) error

    // Monitoring
    GetMetrics(deviceID string) (*GPUMetrics, error)
}

// Implementations:
// - CUDABackend:  NVIDIA GPUs via CUDA Runtime API
// - ROCmBackend:  AMD GPUs via HIP/ROCm
// - oneAPIBackend: Intel GPUs via Level Zero / SYCL
// - MLXBackend:   Apple Silicon via MLX framework
// - SYCLBackend:  Cross-platform via SYCL runtime
```

GPU Sharing Modes:

Mode	Isolation	Overhead	Use Case
EXCLUSIVE	Full	None	Training jobs, benchmarking
MPS	Process-level	~1%	Inference serving, multiple clients
TIME_SLICE	None	Context switch	Development, testing
MIG	Hardware (NVIDIA only)	None	A100/H100 only

Capability Matching Example:

```

# Job requests:
gpu_requirements:
  count: 2
  capabilities: "(TARGET.VENDOR == 'NVIDIA' || TARGET.VENDOR == 'AMD') &&
                TARGET.MEMORY >= 8589934592 &&
                TARGET.API.CUDA >= '12.0'"
  rank: "TARGET.MEMORY * 0.7 + TARGET.COMPUTE_UNITS * 0.3"
  sharing: MPS

# Node advertises:
gpu_capabilities:
  - vendor: NVIDIA
    model: RTX 4080
    memory: 17179869184 # 16 GB
    compute_units: 76
    api: CUDA 12.4
    features: [tensor_cores, ray_tracing, nvenc]

```

5.5 Health Monitor & Predictor (HMP)

Purpose: Real-time health monitoring, failure prediction, and self-healing for all cluster nodes and services.

Architecture Pattern: Prometheus metrics + eBPF probes + LSTM forecasting + chaos engineering

Key Components:

```

// Health Score (0-100)
type HealthScore struct {
  NodeID      string      `json:"node_id"`
  Overall     int           `json:"overall"` // 0-100 composite score
  Components  ComponentScores `json:"components"`
  Predictions []FailurePrediction `json:"predictions"`
  UpdatedAt   time.Time      `json:"updated_at"`
}

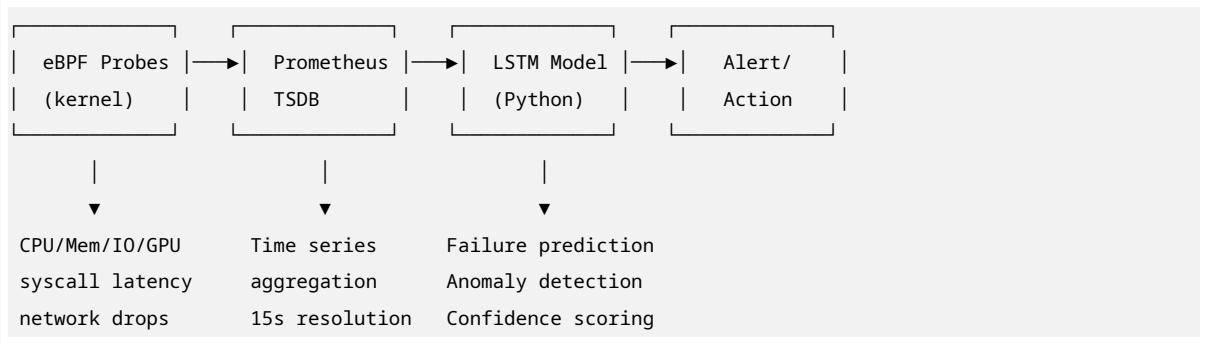
type ComponentScores struct {
  CPU       int `json:"cpu"` // 0-100
  Memory    int `json:"memory"`
  Disk      int `json:"disk"`
  Network   int `json:"network"`
  GPU       int `json:"gpu"`
  Temperature int `json:"temperature"`
  Services  int `json:"services"`
}

type FailurePrediction struct {
  Component  string      `json:"component"` // "memory", "disk", "gpu"
  Probability float64     `json:"probability"` // 0.0 - 1.0
  Horizon    time.Duration `json:"horizon"` // Predicted within this window
}

```

```
Severity      Severity  `json:"severity"`      // INFO, WARNING, CRITICAL
RecommendedAction string `json:"recommended_action"`
}
```

Monitoring Pipeline:



Self-Healing Actions:

Trigger	Condition	Action	Auto-Approved
MemoryPressure	Available < 5%	Migrate largest session	Yes
GPUPanic	ECC errors > threshold	Mark GPU unhealthy, migrate workloads	Yes
DiskFull	Available < 10%	Clean temp files, alert if persists	Yes
NodeUnhealthy	Health score < 30 for 5min	Migrate all sessions, mark FAILED	Yes
PredictedFailure	Probability > 0.8 within 24h	Proactive migration, LLM notifies	No (advisory)
NetworkPartition	Phi accrual > 8	Quarantine node, verify via alternative path	Yes

5.6 LLM Brain (Advisory Controller)

Purpose: Self-tuning advisory system that learns from cluster behavior and suggests optimizations. Never makes binding decisions.

Architecture Pattern: RAG + Constitutional AI + LLMsVerifier + Policy Engine

Key Components:

```
type LLMBrain struct {
    // Configuration (loaded from HelixConstitution)
    Constitution []Principle `json:"constitution"`
    SafetyConstraints []Constraint `json:"safety_constraints"`

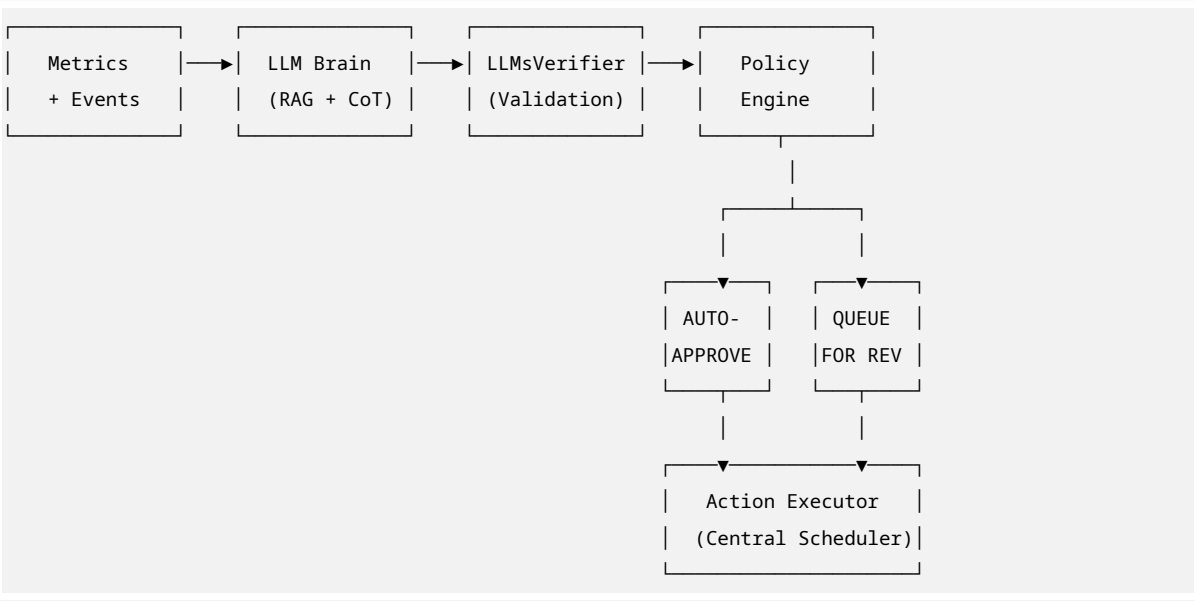
    // State
    KnowledgeBase *RAGStore `json:"- "` // Retrieval-Augmented Generation
    Memory *ConversationMemory `json:"- "` // Short-term context
}
```

```
LearnedPolicies []Policy      `json:"learned_policies"`

// Verification
Verifier      *LLMsVerifier `json:"- "`      // Mandatory verification
PolicyEngine  *OPAEngine   `json:"- "`      // Open Policy Agent
}

type Advisory struct {
    ID          string      `json:"id"`
    Type        AdvisoryType `json:"type"`      // MIGRATION, SCALING, CONFIG, ALERT
    Description string      `json:"description"`
    Rationale   string      `json:"rationale"` // Chain-of-thought reasoning
    ProposedAction ActionSpec `json:"proposed_action"`
    Confidence  float64     `json:"confidence"` // 0.0 - 1.0
    RiskLevel   RiskLevel  `json:"risk_level"` // LOW, MEDIUM, HIGH, CRITICAL
    AutoApprove bool        `json:"auto_approve"` // Based on risk + policy
    Status      AdvisoryStatus `json:"status"`     // PENDING, APPROVED, REJECTED, APPLIED
    CreatedAt   time.Time   `json:"created_at"`
}
```

Decision Flow:



Constitutional Constraints (from HelixConstitution):

```
principles:
- id: SAFETY_FIRST
  text: "Never propose actions that could compromise data integrity or cluster stability"
  priority: 1
- id: NO_BLUFF
  text: "Do not propose actions with confidence below the evidence threshold"
  priority: 2
- id: GRACEFUL_DEGRADATION
  text: "Always prefer actions that reduce capacity over actions that risk correctness"
  priority: 3
- id: TRANSPARENCY
```

```
text: "Always provide full chain-of-thought rationale for every proposal"
priority: 4
- id: HUMAN_IN_LOOP
text: "Critical actions (risk >= HIGH) require human approval"
priority: 5
```

6. MICROSERVICES SPECIFICATION

Service Catalog

#	Service	Language	Port	Replicas	Depends On
1	API Gateway	Go	8443	2+	All services
2	Node Discovery	Go	8081	3 (Raft)	etcd
3	Resource Scheduler	Go	8082	2+	etcd, Node Discovery
4	Session Manager	Go	8083	2+	etcd, Scheduler
5	GPU Compute	Go + C	8084	1 per GPU node	Scheduler, Node Agent
6	Health Monitor	Go + Python	8085	2	Prometheus, all services
7	LLM Brain	Go	8086	1+	LLMsVerifier, Policy Engine
8	Policy Engine	Go (OPA)	8087	2	etcd
9	Security Manager	Go	8088	2	etcd, WireGuard
10	Build Service	Go	8089	2+	Scheduler, Ceph
11	Backup Service	Go	8090	1+	PostgreSQL, Ceph
12	Metrics Collector	Go	8091	1 per node	Prometheus
13	Event Bus	Go	8092	2+	NATS, Kafka
14	Setup Wizard	BASH + Go	8093	1 (ephemeral)	—

Service Communication Matrix

	ND	RS	SM	GC	HM	LB	PE	SM	BS	BK	MC	EB	SW
API Gateway	←	←	←	←	←	←	←	←	←	←	←	←	←
Node Discovery	→				→		→	→			→	→	
Resource Scheduler	→	→		→	→	→	→				→	→	
Session Manager				→		→		→			→		
GPU Compute	→	→	→				→		→		→		
Health Monitor	→	→	→	→	→	→	→	→	→	→	→	→	
LLM Brain	→	→	→	→	→		→	→	→	→	→	→	
Policy Engine	→	→	→	→	→	→		→	→	→	→	→	
Security Manager	→	→	→	→	→	→	→		→	→	→	→	
Build Service	→	→	→			→		→		→		→	
Backup Service	→					→		→	→		→	→	
Metrics Collector	→	→	→	→		→		→	→	→	→	→	
Event Bus	→	→	→	→	→	→	→	→	→	→	→		→

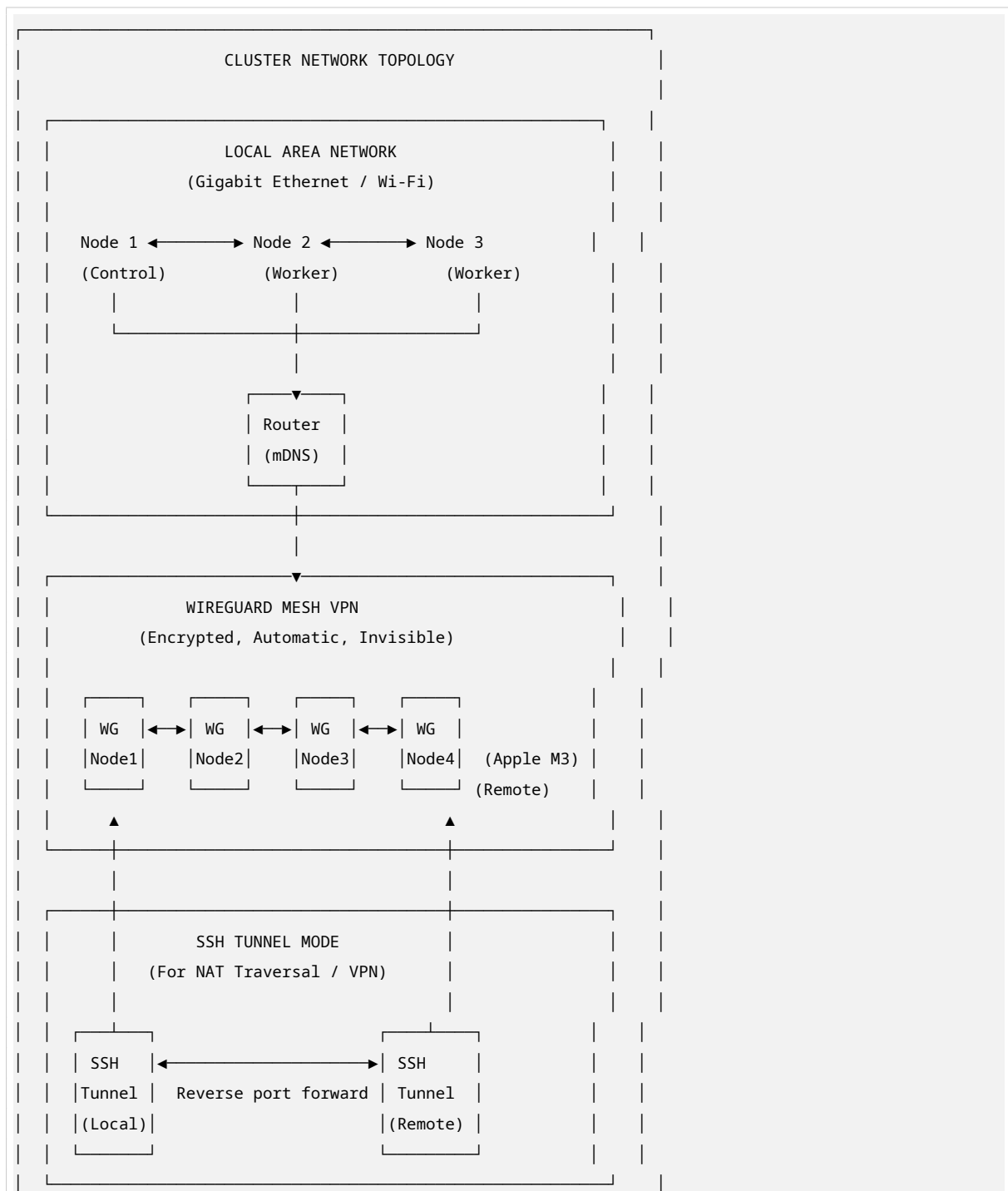
→ → → → → → → → → → → →

Legend: $\rightarrow$ = calls/service dependency, $\leftarrow$ = serves requests

7. NETWORK ARCHITECTURE

7.1 Topology

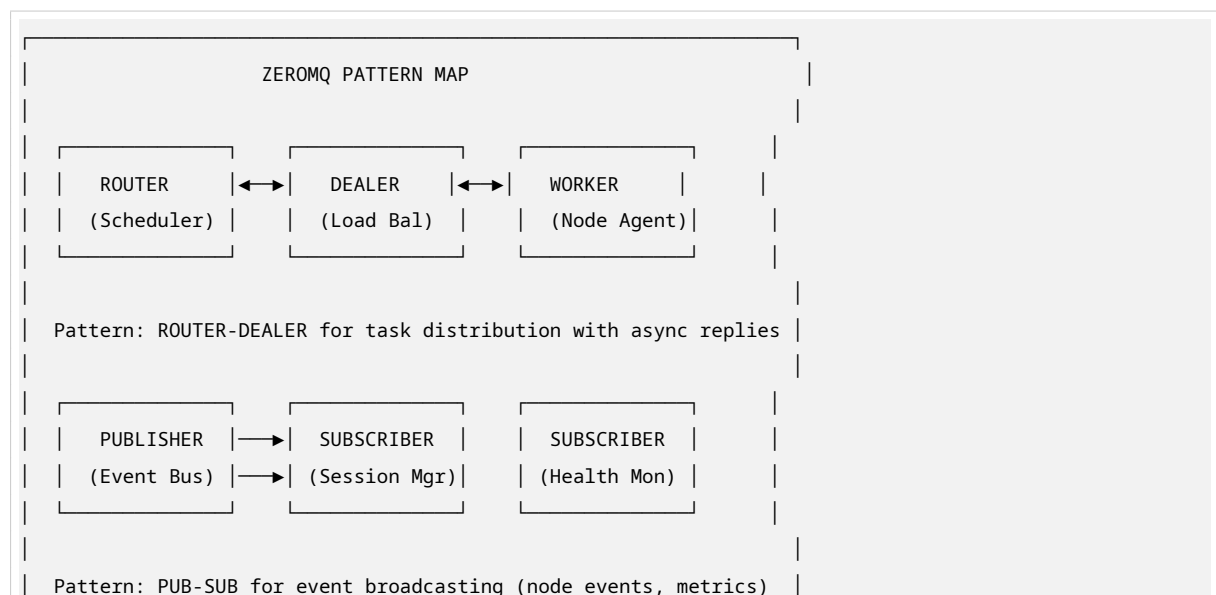
The Cluster OS supports three network modes that can operate simultaneously:

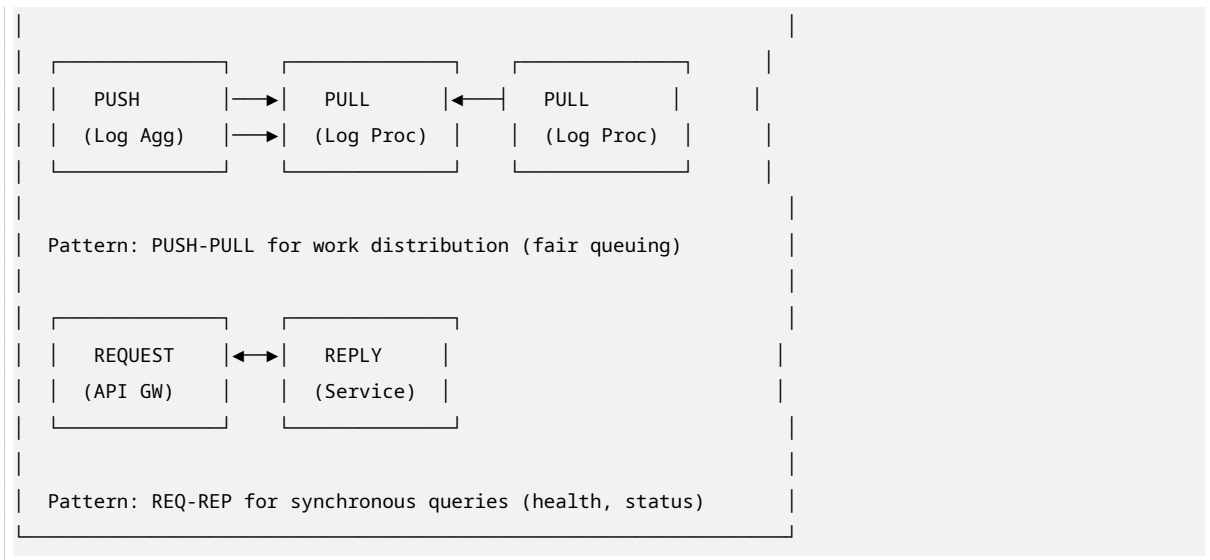


7.2 Communication Protocols by Purpose

Purpose	Protocol	Port	Encryption	Notes
Control messages	NATS	4222	WireGuard tunnel	Sub-millisecond latency
Service RPC	gRPC (HTTP/2)	8443	mTLS	Structured API calls
Data streaming	Apache Arrow Flight	47470	mTLS	Zero-copy data transfer
Real-time I/O	WebSocket	8443 (upgraded)	WSS	Terminal/ session streaming
Event log	Kafka	9092	WireGuard tunnel	Audit trail, analytics
Metrics	Prometheus scrape	9090	WireGuard tunnel	15s scrape interval
Health checks	HTTP/1.1	8080	WireGuard tunnel	Liveness/ readiness probes
VPN mesh	WireGuard UDP	51820	ChaCha20-Poly1305	Kernel or userspace
SSH tunnel	SSH	22	AES-256-GCM	Fallback for NAT traversal
Discovery	mDNS	5353/UDP	None (local only)	Local network discovery
etcd	etcd client	2379	mTLS	Cluster state consensus

7.3 ZeroMQ Message Patterns





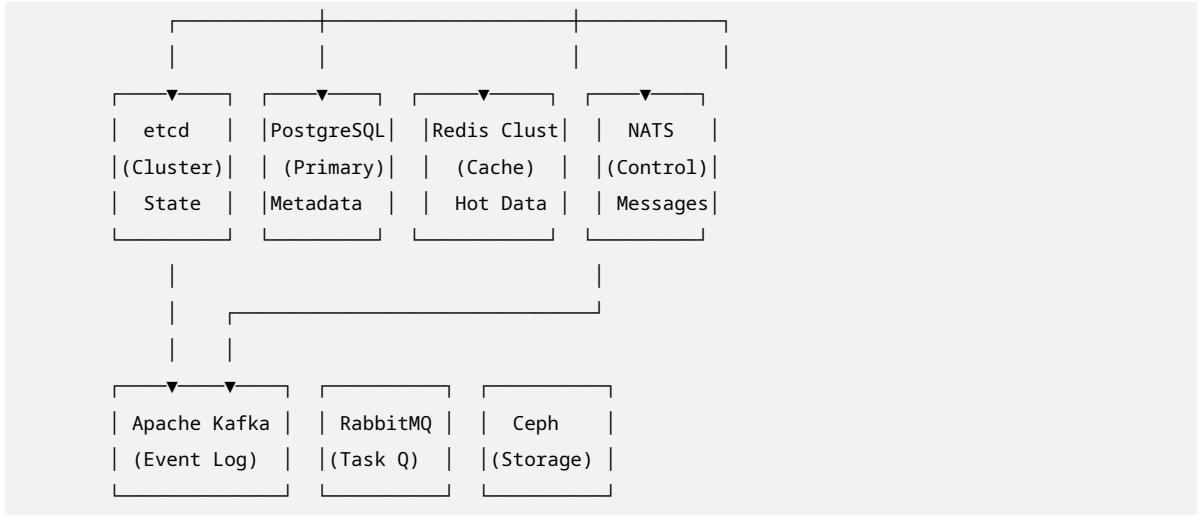
8. DATA ARCHITECTURE

8.1 Data Stores

Store	Technology	Consistency	Use Case
Cluster State	etcd (Raft)	Strong (CP)	Node registry, scheduler state, locks
Primary Metadata	PostgreSQL 16+	Strong ACID	Sessions, users, audit logs
Distributed Cache	Redis Cluster 7+	Eventual	Session state, hot data, rate limiting
Per-Node State	dqlite (SQLite+Raft)	Strong (local)	Node config, local metrics, offline data
Event Log	Apache Kafka 4.0 (KRaft)	Eventual	Audit trail, event sourcing, replay
Object Storage	Ceph RGW/RADOS	Eventual	Build artifacts, checkpoints, backups
Filesystem	CephFS	POSIX-ish	Shared storage across cluster
Task Queue	RabbitMQ	At-least-once	Build jobs, batch processing
Control Messages	NATS + JetStream	At-least-once	Real-time cluster coordination

8.2 Data Flow Diagram





8.3 Caching Strategy

```
// Multi-layer cache architecture
type CacheManager struct {
    L1 *LocalCache    // In-process, per-service (ristretto)
    L2 *RedisCache    // Shared, sub-ms (Redis Cluster)
    L3 *DiskCache     // Local SSD fallback (badgerDB)
}

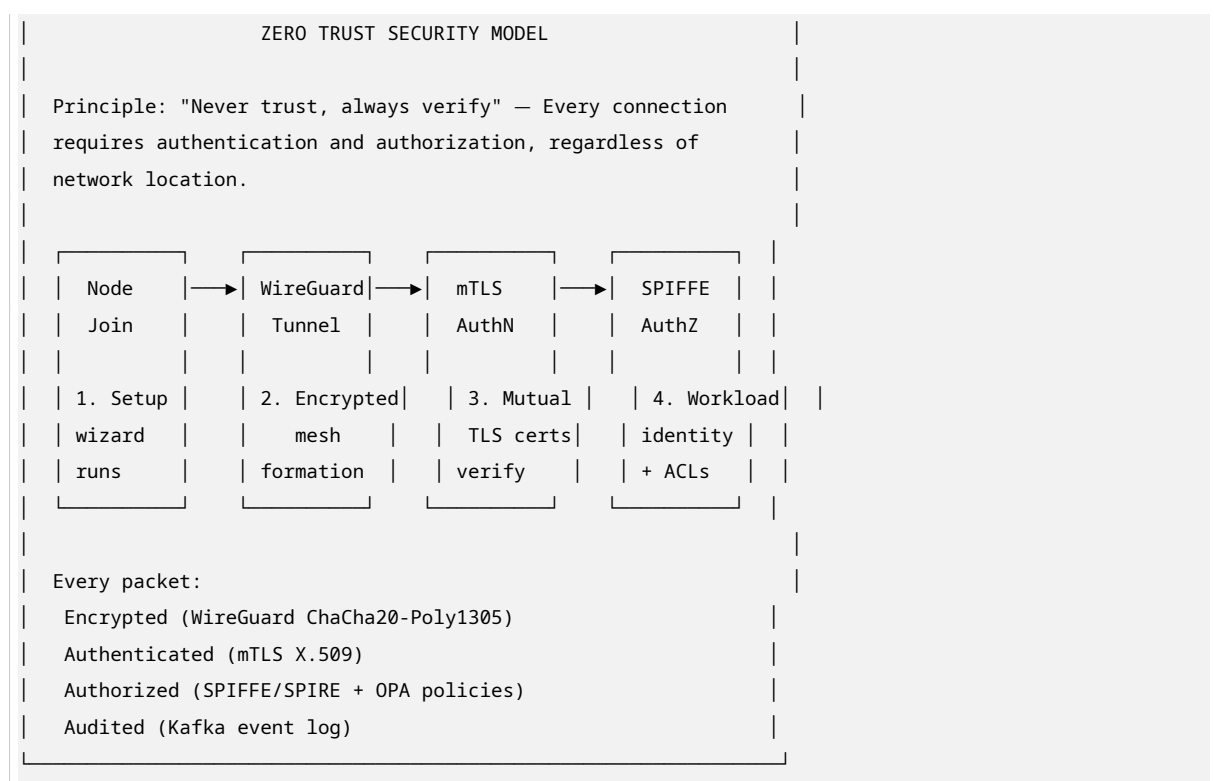
type CachePolicy struct {
    Key      string
    TTL      time.Duration
    InvalidateOn []string // Event types that invalidate this cache
    WarmOnStart bool      // Pre-populate on startup
    Compression bool      // Compress large values
}

// Cache layers per data type:
// | Data Type | L1 | L2 | L3 |
// |-----|-----|-----|-----|
// | Session state | | (CRDT) | |
// | Node resources | | | |
// | GPU metrics | | | |
// | Build artifacts | | | (CAS) |
// | User auth | | | |
// | Scheduler state | | | |
// |-----|-----|-----|-----|
```

9. SECURITY ARCHITECTURE

9.1 Zero Trust Model

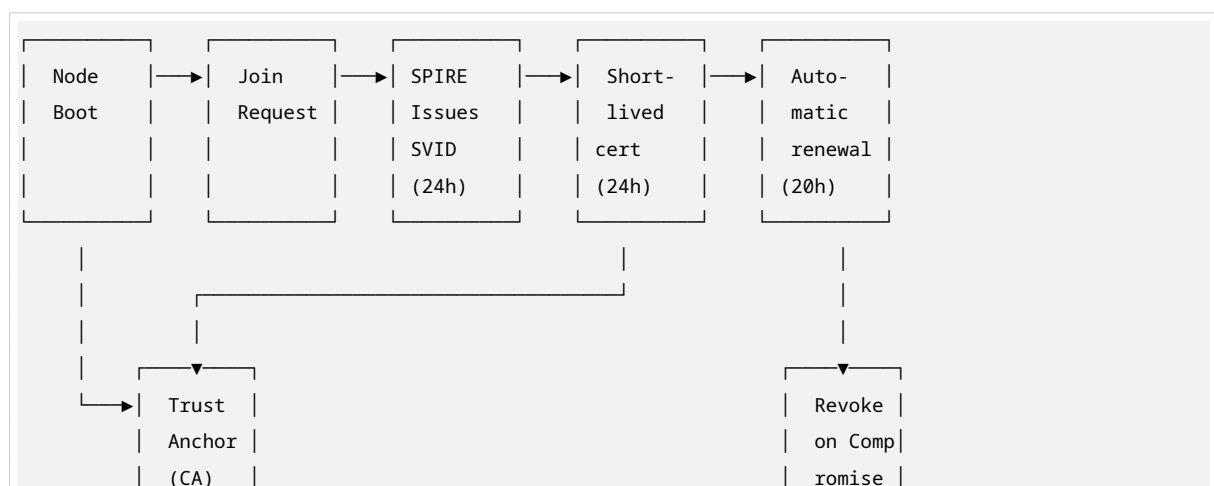




9.2 Security Layers

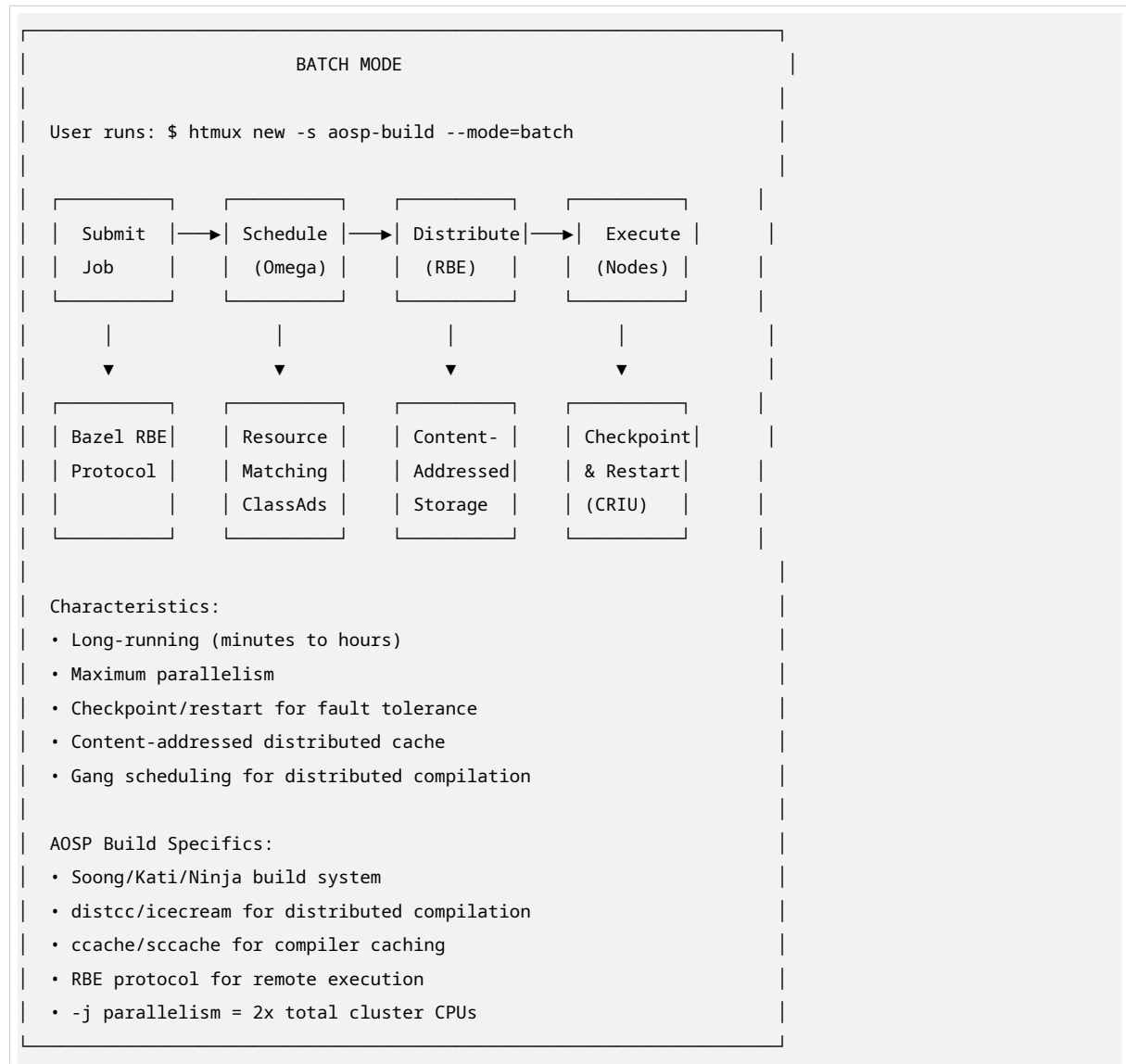
Layer	Technology	Purpose
Transport	WireGuard	Encrypted mesh between all nodes
Service AuthN	mTLS (SPIFFE)	Certificate-based service identity
User AuthN	OIDC (Google/GitHub)	User authentication
AuthZ	OPA / HelixConstitution	Policy-based access control
Secrets	HashiCorp Vault	Secret storage and rotation
Node Attestation	SPIRE	Verify node identity on join
Runtime	seccomp + AppArmor	System call filtering
Audit	Kafka + PostgreSQL	Immutable audit log

9.3 Certificate Lifecycle

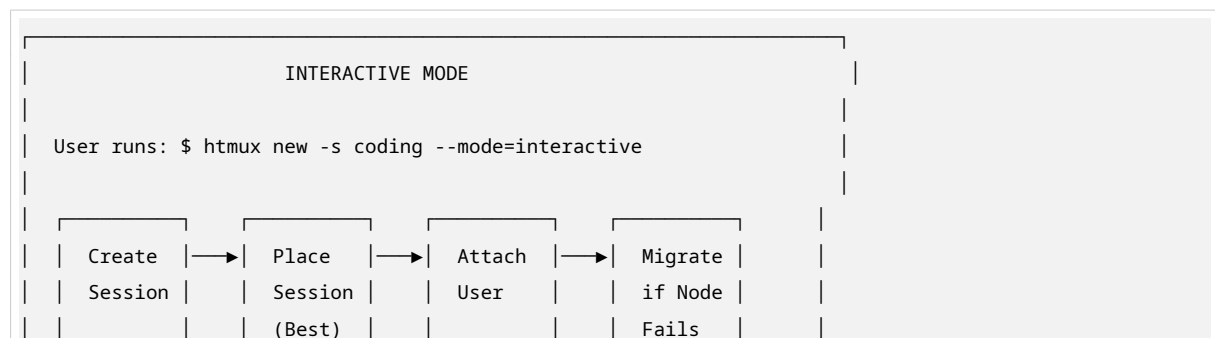


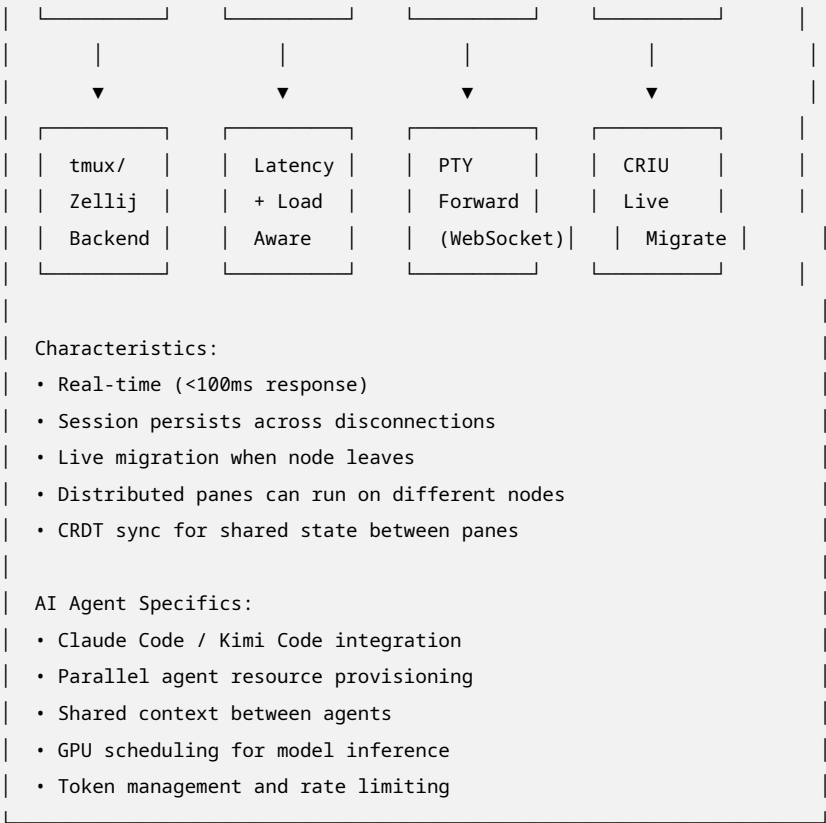
10. EXECUTION MODES

10.1 Batch Mode (AOSP Builds, Data Processing)



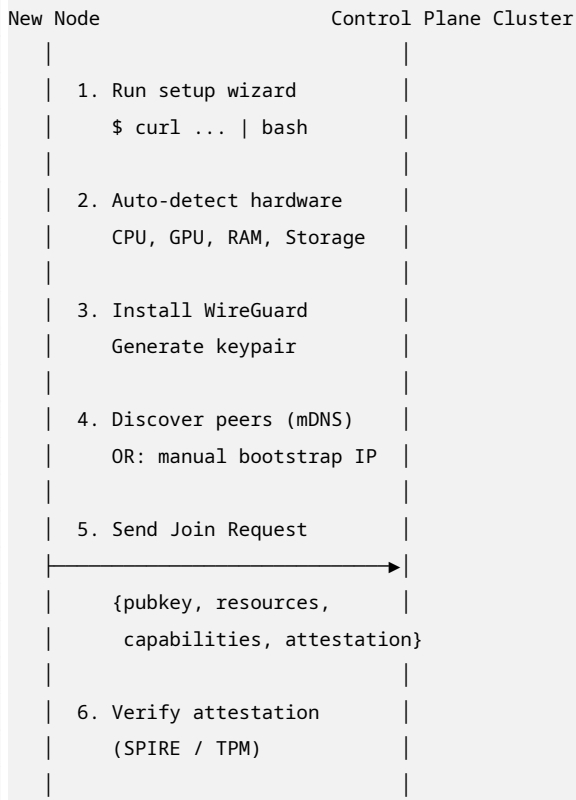
10.2 Interactive Mode (AI Agents, Development)

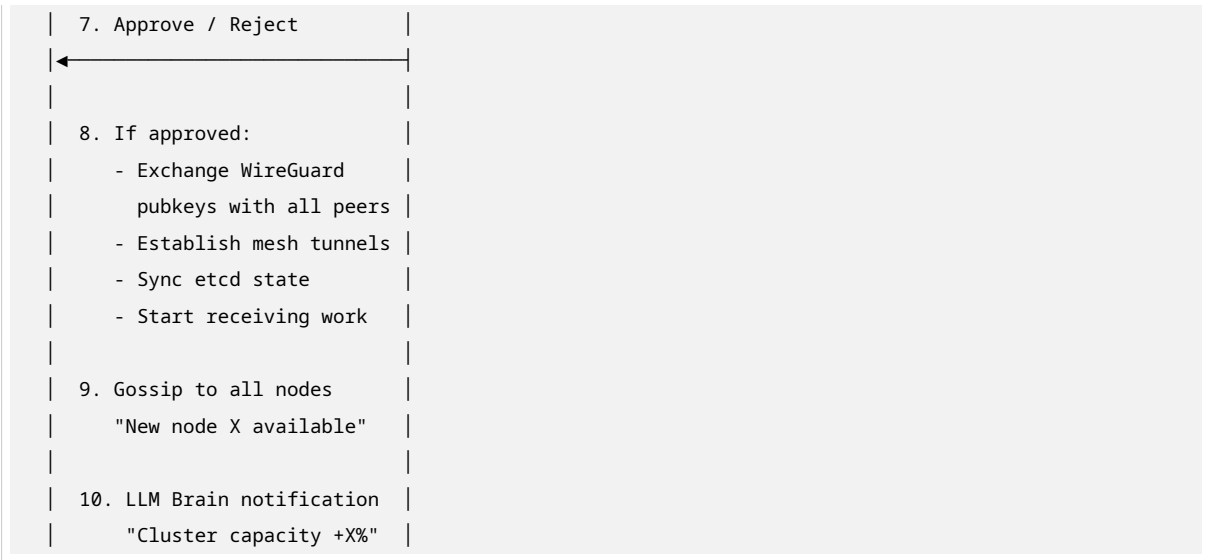




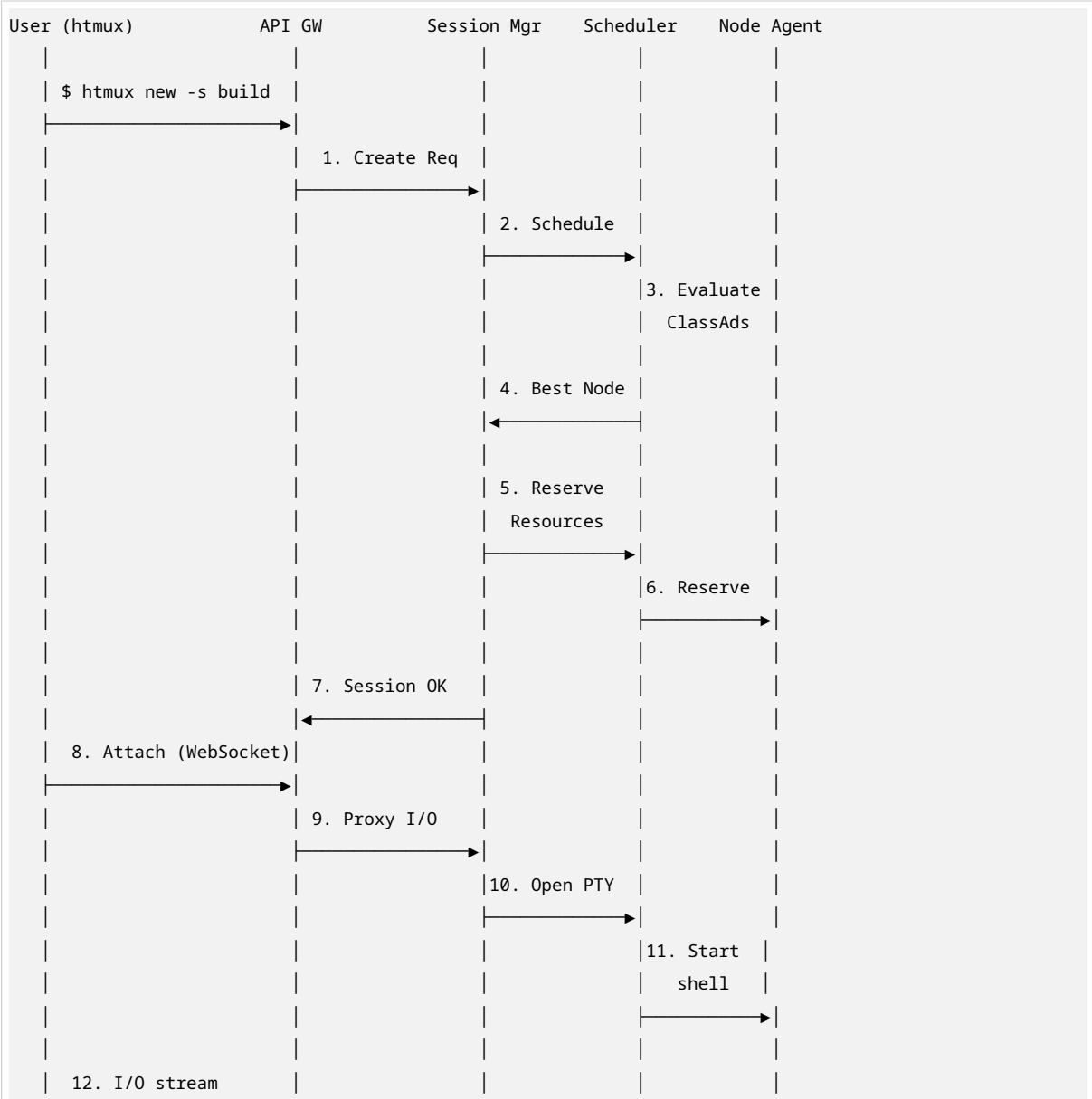
11. COMPONENT INTERACTION FLOWS

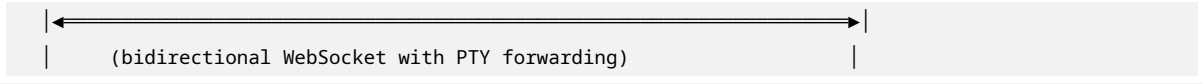
11.1 Node Join Flow



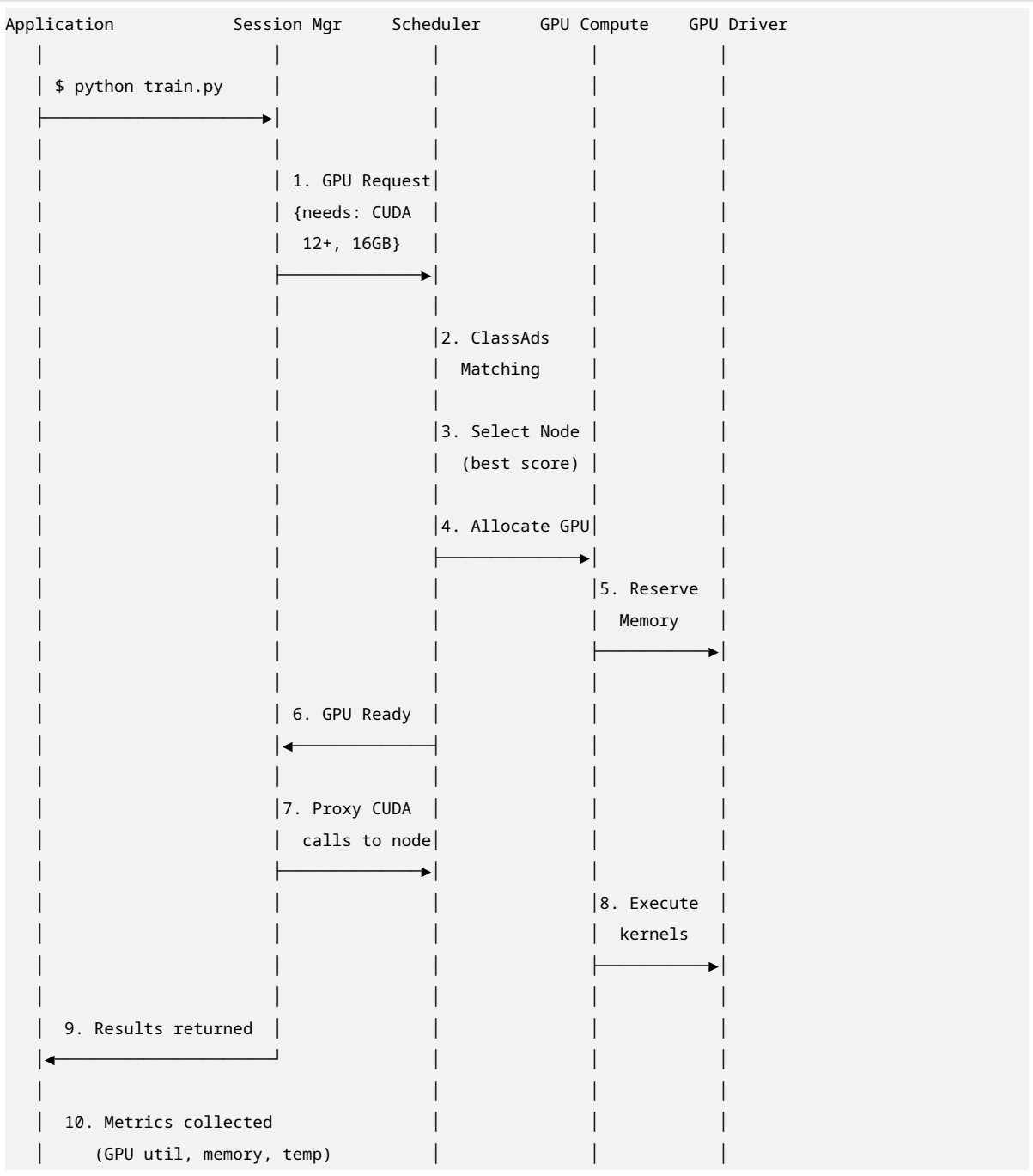


11.2 Session Create & Execute Flow





11.3 GPU Job Execution Flow



12. DATABASE SCHEMAS

12.1 PostgreSQL Primary Schema

```
-- =====
-- HELIX CLUSTER OS — POSTGRESQL PRIMARY SCHEMA
```

```

-- =====
-- Node Registry (shadow of etcd, for queries and history)
CREATE TABLE nodes (
    id                UUID PRIMARY KEY,
    hostname           VARCHAR(255) NOT NULL,
    ip_addresses       INET[] NOT NULL,
    wg_pubkey          TEXT NOT NULL UNIQUE,
    spiffe_id          TEXT NOT NULL UNIQUE,
    status             VARCHAR(20) NOT NULL DEFAULT 'JOINING',
    role              VARCHAR(20) NOT NULL DEFAULT 'WORKER',
    cpu_arch           VARCHAR(20) NOT NULL,
    cpu_cores          INT NOT NULL,
    cpu_threads        INT NOT NULL,
    memory_bytes       BIGINT NOT NULL,
    gpu_count          INT NOT NULL DEFAULT 0,
    storage_bytes      BIGINT NOT NULL,
    labels             JSONB DEFAULT '{}',
    region             VARCHAR(100),
    version            VARCHAR(50) NOT NULL,
    joined_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_seen          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    left_at            TIMESTAMPTZ,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_nodes_status ON nodes(status);
CREATE INDEX idx_nodes_role ON nodes(role);
CREATE INDEX idx_nodes_region ON nodes(region);
CREATE INDEX idx_nodes_labels ON nodes USING GIN(labels);

-- GPU Devices
CREATE TABLE gpu_devices (
    id                UUID PRIMARY KEY,
    node_id           UUID NOT NULL REFERENCES nodes(id) ON DELETE CASCADE,
    vendor            VARCHAR(20) NOT NULL,
    model            VARCHAR(100) NOT NULL,
    driver_version    VARCHAR(50) NOT NULL,
    api              VARCHAR(20) NOT NULL,
    api_version       VARCHAR(20) NOT NULL,
    total_memory      BIGINT NOT NULL,
    compute_units     INT NOT NULL,
    features          TEXT[] DEFAULT '{}',
    attributes        JSONB DEFAULT '{}',
    status            VARCHAR(20) NOT NULL DEFAULT 'AVAILABLE',
    allocated_to      UUID, -- session ID
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE INDEX idx_gpu_node ON gpu_devices(node_id);
CREATE INDEX idx_gpu_status ON gpu_devices(status);
CREATE INDEX idx_gpu_vendor ON gpu_devices(vendor);

-- Sessions
CREATE TABLE sessions (
  id          UUID PRIMARY KEY,
  name        VARCHAR(255) NOT NULL,
  owner       TEXT NOT NULL, -- SPIFFE ID
  status      VARCHAR(20) NOT NULL DEFAULT 'CREATING',
  mode        VARCHAR(20) NOT NULL DEFAULT 'INTERACTIVE',
  backend     VARCHAR(20) NOT NULL DEFAULT 'TMUX',
  backend_id  TEXT,
  node_id     UUID REFERENCES nodes(id),
  cpu_request INT NOT NULL DEFAULT 1000, -- millicores
  memory_request BIGINT NOT NULL DEFAULT 1073741824, -- 1GB
  gpu_request JSONB DEFAULT NULL,
  priority    INT NOT NULL DEFAULT 50,
  labels      JSONB DEFAULT '{}',
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  started_at  TIMESTAMPTZ,
  terminated_at TIMESTAMPTZ,
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_sessions_owner ON sessions(owner);
CREATE INDEX idx_sessions_status ON sessions(status);
CREATE INDEX idx_sessions_node ON sessions(node_id);
CREATE INDEX idx_sessions_mode ON sessions(mode);

-- Windows (within sessions)
CREATE TABLE session_windows (
  id          UUID PRIMARY KEY,
  session_id  UUID NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
  name        VARCHAR(255) NOT NULL,
  layout      VARCHAR(50) NOT NULL DEFAULT 'tiled',
  active      BOOLEAN NOT NULL DEFAULT FALSE,
  crdt_state  JSONB DEFAULT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Panes (within windows)
CREATE TABLE session_panes (
  id          UUID PRIMARY KEY,
  window_id   UUID NOT NULL REFERENCES session_windows(id) ON DELETE CASCADE,
  node_id     UUID REFERENCES nodes(id),
  command     TEXT,
  working_dir TEXT,
  environment JSONB DEFAULT '{}',
  cpu_limit   INT,
  memory_limit BIGINT,

```

```

    gpu_id          UUID REFERENCES gpu_devices(id),
    status          VARCHAR(20) NOT NULL DEFAULT 'CREATING',
    crdt_state      JSONB DEFAULT NULL,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Resource Reservations
CREATE TABLE reservations (
    id              UUID PRIMARY KEY,
    session_id      UUID NOT NULL REFERENCES sessions(id),
    node_id         UUID NOT NULL REFERENCES nodes(id),
    cpu_millicores  INT NOT NULL,
    memory_bytes    BIGINT NOT NULL,
    gpu_ids         UUID[],
    status          VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    expires_at      TIMESTAMPTZ NOT NULL,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_reservations_session ON reservations(session_id);
CREATE INDEX idx_reservations_node ON reservations(node_id);
CREATE INDEX idx_reservations_status ON reservations(status);
CREATE INDEX idx_reservations_expires ON reservations(expires_at);

-- Migration History
CREATE TABLE migration_history (
    id              UUID PRIMARY KEY,
    session_id      UUID NOT NULL REFERENCES sessions(id),
    source_node     UUID NOT NULL REFERENCES nodes(id),
    target_node     UUID NOT NULL REFERENCES nodes(id),
    method          VARCHAR(20) NOT NULL, -- CRIU, DMTCP, RESTART
    duration_ms     INT NOT NULL,
    data_size_bytes BIGINT,
    success         BOOLEAN NOT NULL,
    error_message   TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_migrations_session ON migration_history(session_id);

-- Audit Log (immutable)
CREATE TABLE audit_log (
    id              BIGSERIAL PRIMARY KEY,
    timestamp       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    event_type      VARCHAR(50) NOT NULL,
    severity        VARCHAR(10) NOT NULL DEFAULT 'INFO',
    actor           TEXT NOT NULL, -- SPIFFE ID or 'system'
    resource_type   VARCHAR(50) NOT NULL,
    resource_id     TEXT,
    action          VARCHAR(50) NOT NULL,

```

```

        details          JSONB DEFAULT '{}',
        source_ip        INET,
        session_id       UUID
    ) PARTITION BY RANGE (timestamp);

-- Create monthly partitions
CREATE TABLE audit_log_2026_06 PARTITION OF audit_log
    FOR VALUES FROM ('2026-06-01') TO ('2026-07-01');
CREATE TABLE audit_log_2026_07 PARTITION OF audit_log
    FOR VALUES FROM ('2026-07-01') TO ('2026-08-01');
-- ... auto-create future partitions

CREATE INDEX idx_audit_time ON audit_log(timestamp);
CREATE INDEX idx_audit_event ON audit_log(event_type);
CREATE INDEX idx_audit_actor ON audit_log(actor);
CREATE INDEX idx_audit_resource ON audit_log(resource_type, resource_id);

-- Users (shadow of OIDC provider)
CREATE TABLE users (
    id            UUID PRIMARY KEY,
    spiffe_id     TEXT NOT NULL UNIQUE,
    email         TEXT,
    name          VARCHAR(255),
    role          VARCHAR(20) NOT NULL DEFAULT 'USER',
    quota_cpu     INT NOT NULL DEFAULT 8000, -- millicores
    quota_memory  BIGINT NOT NULL DEFAULT 17179869184, -- 16GB
    quota_gpu     INT NOT NULL DEFAULT 0,
    labels        JSONB DEFAULT '{}',
    last_login    TIMESTAMPTZ,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_users_spiffe ON users(spiffe_id);

-- Health Snapshots
CREATE TABLE health_snapshots (
    id            BIGSERIAL PRIMARY KEY,
    node_id       UUID NOT NULL REFERENCES nodes(id),
    overall_score INT NOT NULL, -- 0-100
    cpu_score     INT NOT NULL,
    memory_score  INT NOT NULL,
    disk_score    INT NOT NULL,
    network_score INT NOT NULL,
    gpu_score     INT NOT NULL,
    temperature_score INT NOT NULL,
    services_score INT NOT NULL,
    predictions   JSONB DEFAULT '[]',
    metrics       JSONB NOT NULL DEFAULT '{}',
    recorded_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE INDEX idx_health_node ON health_snapshots(node_id);
CREATE INDEX idx_health_time ON health_snapshots(recorded_at);
CREATE INDEX idx_health_score ON health_snapshots(overall_score);

-- LLM Advisories
CREATE TABLE llm_advisories (
    id          UUID PRIMARY KEY,
    type        VARCHAR(20) NOT NULL,
    description  TEXT NOT NULL,
    rationale    TEXT NOT NULL,
    proposed_action JSONB NOT NULL,
    confidence   FLOAT NOT NULL,
    risk_level   VARCHAR(10) NOT NULL,
    auto_approve BOOLEAN NOT NULL DEFAULT FALSE,
    status       VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    applied_by   TEXT, -- SPIFFE ID if manual
    applied_at   TIMESTAMPTZ,
    result       JSONB,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_advisories_status ON llm_advisories(status);
CREATE INDEX idx_advisories_type ON llm_advisories(type);
CREATE INDEX idx_advisories_risk ON llm_advisories(risk_level);

-- =====
-- TRIGGERS AND FUNCTIONS
-- =====

-- Auto-update updated_at timestamp
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER nodes_updated_at BEFORE UPDATE ON nodes
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
CREATE TRIGGER sessions_updated_at BEFORE UPDATE ON sessions
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
-- ... (apply to all tables with updated_at)

-- Audit log trigger
CREATE OR REPLACE FUNCTION audit_trigger()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO audit_log (event_type, severity, actor, resource_type, resource_id, action, details)
    VALUES (

```

```

        TG_OP || '_' || TG_TABLE_NAME,
        CASE WHEN TG_OP = 'DELETE' THEN 'WARNING' ELSE 'INFO' END,
        COALESCE(current_setting('app.current_user', true), 'system'),
        TG_TABLE_NAME,
        NEW.id::text,
        TG_OP,
        to_jsonb(NEW)
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER nodes_audit AFTER INSERT OR UPDATE OR DELETE ON nodes
    FOR EACH ROW EXECUTE FUNCTION audit_trigger();

```

12.2 etcd Key Structure

```

/clusters/
├─ nodes/
│   ├─ {node_id}          → Node (JSON)
│   ├─ {node_id}/status   → NodeStatus
│   ├─ {node_id}/heartbeat → Timestamp
│   └─ {node_id}/leases/   → Resource leases
├─ sessions/
│   ├─ {session_id}       → Session (JSON)
│   ├─ {session_id}/status → SessionStatus
│   └─ {session_id}/routing → I/O routing table
├─ scheduler/
│   ├─ pool/              → ResourcePool (JSON)
│   ├─ queue/             → Pending requests
│   ├─ reservations/      → Active reservations
│   └─ bindings/          → Session-Node bindings
├─ security/
│   ├─ spiffe_ids/        → SPIFFE → Node mapping
│   ├─ wireguard/
│   │   ├─ peers/         → Allowed IPs and pubkeys
│   │   └─ subnets/      → Allocated subnets
│   └─ acl/               → Access control policies
├─ config/
│   ├─ cluster/           → Cluster-wide settings
│   ├─ scheduler/         → Scheduler configuration
│   └─ limits/            → Resource quotas
└─ locks/
    ├─ scheduler/         → Scheduling mutex
    ├─ migrations/        → Migration mutex
    └─ config/            → Config changes mutex

```

12.3 Redis Key Structure

```

# Session state (CRDT-synced, short TTL)
clusters:session:{id}:state → JSON (with vector clock)

```

```

clusteros:session:{id}:routing      → Node routing table
clusteros:session:{id}:windows     → Window list
clusteros:session:{id}:panes       → Pane list

# Hot data (frequently accessed)
clusteros:node:{id}:resources      → Current resource availability
clusteros:node:{id}:health         → Latest health snapshot
clusteros:node:{id}:metrics        → Last 5 min of metrics

# GPU status
clusteros:gpu:{id}:status           → AVAILABLE, ALLOCATED, UNHEALTHY
clusteros:gpu:{id}:metrics          → Temperature, utilization, memory

# Cache
clusteros:cache:sessions            → Session list (sorted by activity)
clusteros:cache:pool                → Resource pool snapshot
clusteros:cache:capabilities        → All cluster capabilities

# Rate limiting
clusteros:ratelimit:{user_id}      → Token bucket counters
clusteros:ratelimit:global          → Global rate limit

# Pub/Sub channels
clusteros:events:nodes              → Node join/leave/fail events
clusteros:events:sessions           → Session create/terminate/migrate
clusteros:events:scheduler          → Scheduling decisions
clusteros:events:alerts             → Health alerts and predictions

```

13. API SPECIFICATIONS

13.1 REST API (Go + Gin Gonic)

```

openapi: 3.0.0
info:
  title: Helix Cluster OS API
  version: 1.0.0
servers:
  - url: https://{control-plane}:8443/v1

# Authentication: mTLS (SPIFFE ID in client certificate)
security:
  - mtls: []

paths:
  # === NODES ===
  /nodes:
    get:
      summary: List cluster nodes
      parameters:

```



```

    - name: status
      in: query
      schema: { type: string, enum: [JOINING, ACTIVE, SUSPECT, LEFT, FAILED] }
    - name: role
      in: query
      schema: { type: string, enum: [WORKER, CONTROL, HYBRID] }
    - name: region
      in: query
      schema: { type: string }
  responses:
    200:
      description: Node list
      content:
        application/json:
          schema:
            type: object
            properties:
              nodes: { type: array, items: { $ref: '#/components/schemas/Node' } }
              total: { type: integer }

/nodes/{id}:
  get:
    summary: Get node details
    parameters:
      - name: id
        in: path
        required: true
        schema: { type: string, format: uuid }
    responses:
      200:
        description: Node details
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Node' }
  delete:
    summary: Remove node from cluster
    responses:
      202: { description: Node removal initiated }

/nodes/{id}/heartbeat:
  post:
    summary: Node heartbeat
    requestBody:
      content:
        application/json:
          schema:
            type: object
            properties:
              health_score: { type: integer }
              resource_usage: { $ref: '#/components/schemas/ResourceUsage' }
              metrics: { type: object }

```

```

    responses:
      200: { description: Heartbeat acknowledged }

/nodes/join:
  post:
    summary: Join cluster (setup wizard)
    requestBody:
      content:
        application/json:
          schema:
            type: object
            required: [pubkey, resources, capabilities]
            properties:
              pubkey: { type: string }
              resources: { $ref: '#/components/schemas/NodeResources' }
              capabilities: { type: array, items: { $ref: '#/components/schemas/Capability' } }
              attestation: { type: object }
    responses:
      201:
        description: Node joined
        content:
          application/json:
            schema:
              type: object
              properties:
                node_id: { type: string, format: uuid }
                cluster_config: { type: object }
                peer_list: { type: array, items: { type: string } }

# === SESSIONS ===

/sessions:
  get:
    summary: List sessions
    parameters:
      - name: owner
        in: query
        schema: { type: string }
      - name: status
        in: query
        schema: { type: string }
      - name: mode
        in: query
        schema: { type: string, enum: [INTERACTIVE, BATCH] }
    responses:
      200:
        description: Session list
        content:
          application/json:
            schema:
              type: object
              properties:

```

```

        sessions: { type: array, items: { $ref: '#/components/schemas/Session' } }
        total: { type: integer }

post:
  summary: Create session
  requestBody:
    content:
      application/json:
        schema:
          type: object
          required: [name, mode]
          properties:
            name: { type: string }
            mode: { type: string, enum: [INTERACTIVE, BATCH] }
            backend: { type: string, enum: [TMUX, ZELLIJ, SCREEN, NATIVE] }
            resources:
              type: object
              properties:
                cpu: { type: integer }
                memory: { type: integer }
                gpu: { $ref: '#/components/schemas/GPURequest' }
            command: { type: string }
            working_dir: { type: string }
            environment: { type: object }
            labels: { type: object }
  responses:
    201:
      description: Session created
      content:
        application/json:
          schema: { $ref: '#/components/schemas/Session' }

/sessions/{id}:
  get:
    summary: Get session details
    responses:
      200:
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Session' }
  delete:
    summary: Terminate session
    responses:
      202: { description: Termination initiated }

/sessions/{id}/attach:
  post:
    summary: Attach to session
    description: Upgrades to WebSocket for I/O streaming
    responses:
      101: { description: WebSocket upgrade }

```

```

/sessions/{id}/detach:
  post:
    summary: Detach from session (session continues)
    responses:
      200: { description: Detached successfully }

/sessions/{id}/migrate:
  post:
    summary: Migrate session to different node
    requestBody:
      content:
        application/json:
          schema:
            type: object
            properties:
              target_node: { type: string, format: uuid }
              method: { type: string, enum: [CRIU, DMTCP, RESTART] }
    responses:
      202:
        description: Migration initiated
        content:
          application/json:
            schema:
              type: object
              properties:
                migration_id: { type: string, format: uuid }

/sessions/{id}/windows:
  get:
    summary: List windows
    responses:
      200:
        content:
          application/json:
            schema:
              type: object
              properties:
                windows: { type: array, items: { $ref: '#/components/schemas/Window' } }

  post:
    summary: Create window
    requestBody:
      content:
        application/json:
          schema:
            type: object
            properties:
              name: { type: string }
              layout: { type: string }
    responses:

```

```

    201: { description: Window created }

/sessions/{id}/windows/{wid}/panes:
  post:
    summary: Create pane (can be on different node)
    requestBody:
      content:
        application/json:
          schema:
            type: object
            properties:
              command: { type: string }
              node_id: { type: string, format: uuid }
              resources:
                type: object
                properties:
                  cpu: { type: integer }
                  memory: { type: integer }
                  gpu: { $ref: '#/components/schemas/GPURequest' }
            responses:
              201: { description: Pane created }

# === RESOURCES ===

/pool:
  get:
    summary: View resource pool
    responses:
      200:
        content:
          application/json:
            schema: { $ref: '#/components/schemas/ResourcePool' }

/pool/utilization:
  get:
    summary: Cluster utilization
    responses:
      200:
        content:
          application/json:
            schema:
              type: object
              properties:
                cpu_percent: { type: number }
                memory_percent: { type: number }
                gpu_percent: { type: number }
                node_count: { type: integer }
                active_sessions: { type: integer }

/schedule:
  post:
    summary: Submit resource request

```

```

requestBody:
  content:
    application/json:
      schema: { $ref: '#/components/schemas/ResourceRequest' }
responses:
  202:
    description: Request queued
    content:
      application/json:
        schema:
          type: object
          properties:
            request_id: { type: string, format: uuid }
            status: { type: string }
            estimated_wait: { type: integer, description: seconds }

# === HEALTH ===
/health:
  get:
    summary: Cluster health overview
    responses:
      200:
        content:
          application/json:
            schema:
              type: object
              properties:
                overall_score: { type: integer }
                node_count: { type: integer }
                healthy_nodes: { type: integer }
                active_sessions: { type: integer }
                alerts: { type: array, items: { $ref: '#/components/schemas/Alert' } }
                predictions: { type: array, items: { $ref: '#/components/schemas/Prediction' } }

/health/nodes/{id}:
  get:
    summary: Node health details
    responses:
      200:
        content:
          application/json:
            schema: { $ref: '#/components/schemas/HealthScore' }

# === ADVISORIES ===
/advisories:
  get:
    summary: List LLM advisories
    parameters:
      - name: status
        in: query
        schema: { type: string, enum: [PENDING, APPROVED, REJECTED, APPLIED] }

```

```

    responses:
      200:
        content:
          application/json:
            schema:
              type: object
              properties:
                advisories: { type: array, items: { $ref: '#/components/schemas/Advisory' } }

/advisories/{id}/approve:
  post:
    summary: Approve advisory
    responses:
      200: { description: Advisory approved and applied }

/advisories/{id}/reject:
  post:
    summary: Reject advisory
    requestBody:
      content:
        application/json:
          schema:
            type: object
            properties:
              reason: { type: string }
    responses:
      200: { description: Advisory rejected }

# === WEBSOCKET ===
/ws/sessions/{id}/stream:
  get:
    summary: Bidirectional I/O stream for session
    description: WebSocket endpoint for terminal I/O
    parameters:
      - name: id
        in: path
        required: true
        schema: { type: string, format: uuid }
    responses:
      101: { description: WebSocket upgrade }

/ws/nodes/watch:
  get:
    summary: Watch node events
    description: WebSocket for real-time node join/leave/fail events
    responses:
      101: { description: WebSocket upgrade }

/ws/pool/watch:
  get:
    summary: Watch resource pool changes

```

```

    description: WebSocket for real-time resource changes
    responses:
      101: { description: WebSocket upgrade }

components:
  schemas:
    Node:
      type: object
      properties:
        id: { type: string, format: uuid }
        hostname: { type: string }
        status: { type: string }
        role: { type: string }
        resources: { $ref: '#/components/schemas/NodeResources' }
        capabilities: { type: array, items: { $ref: '#/components/schemas/Capability' } }
        labels: { type: object }
        joined_at: { type: string, format: date-time }

    NodeResources:
      type: object
      properties:
        cpu_cores: { type: integer }
        cpu_threads: { type: integer }
        memory_bytes: { type: integer }
        gpu_count: { type: integer }
        storage_bytes: { type: integer }

    Capability:
      type: object
      properties:
        name: { type: string }
        type: { type: string }
        version: { type: string }
        quantity: { type: integer }
        attributes: { type: object }

    Session:
      type: object
      properties:
        id: { type: string, format: uuid }
        name: { type: string }
        owner: { type: string }
        status: { type: string }
        mode: { type: string }
        backend: { type: string }
        node_id: { type: string, format: uuid }
        resources: { type: object }
        created_at: { type: string, format: date-time }

    Window:
      type: object

```



```
properties:
  id: { type: string, format: uuid }
  name: { type: string }
  layout: { type: string }
  active: { type: boolean }

ResourcePool:
  type: object
  properties:
    total: { $ref: '#/components/schemas/ResourceSnapshot' }
    available: { $ref: '#/components/schemas/ResourceSnapshot' }
    reserved: { $ref: '#/components/schemas/ResourceSnapshot' }
    utilization: { type: object }

ResourceSnapshot:
  type: object
  properties:
    cpu_shares: { type: integer }
    memory_bytes: { type: integer }
    gpu_devices: { type: array }

ResourceRequest:
  type: object
  properties:
    session_id: { type: string }
    priority: { type: integer }
    requirements: { type: string }
    rank: { type: string }
    resources: { type: object }
    mode: { type: string }

GPURequest:
  type: object
  properties:
    count: { type: integer }
    vendor: { type: string }
    min_memory: { type: integer }
    api: { type: string }
    min_version: { type: string }
    sharing: { type: string }

HealthScore:
  type: object
  properties:
    node_id: { type: string }
    overall: { type: integer }
    components: { type: object }
    predictions: { type: array }

Alert:
  type: object
```

```

properties:
  id: { type: string }
  severity: { type: string }
  message: { type: string }
  node_id: { type: string }
  created_at: { type: string, format: date-time }

Prediction:
  type: object
  properties:
    component: { type: string }
    probability: { type: number }
    horizon: { type: string }
    severity: { type: string }

Advisory:
  type: object
  properties:
    id: { type: string, format: uuid }
    type: { type: string }
    description: { type: string }
    rationale: { type: string }
    confidence: { type: number }
    risk_level: { type: string }
    status: { type: string }

```

13.2 gRPC Services

```

// clusteros.proto
syntax = "proto3";
package clusteros.v1;

import "google/protobuf/timestamp.proto";
import "google/protobuf/duration.proto";
import "google/protobuf/empty.proto";

// === Node Service ===
service NodeService {
  rpc Join(JoinRequest) returns (JoinResponse);
  rpc Heartbeat(HeartbeatRequest) returns (HeartbeatResponse);
  rpc Leave(LeaveRequest) returns (google.protobuf.Empty);
  rpc GetNode(GetNodeRequest) returns (Node);
  rpc ListNodes(ListNodesRequest) returns (stream Node);
  rpc WatchNodes(WatchNodesRequest) returns (stream NodeEvent);
}

message JoinRequest {
  string hostname = 1;
  bytes wireguard_pubkey = 2;
  NodeResources resources = 3;
  repeated Capability capabilities = 4;
}

```

```

    bytes attestation = 5;
}

message JoinResponse {
    string node_id = 1;
    bytes cluster_ca_cert = 2;
    repeated PeerInfo peers = 3;
    ClusterConfig config = 4;
}

// === Session Service ===
service SessionService {
    rpc CreateSession(CreateSessionRequest) returns (Session);
    rpc AttachSession(AttachSessionRequest) returns (stream IOEvent);
    rpc DetachSession(DetachSessionRequest) returns (google.protobuf.Empty);
    rpc TerminateSession(TerminateSessionRequest) returns (google.protobuf.Empty);
    rpc MigrateSession(MigrateSessionRequest) returns (MigrationStatus);
    rpc GetSession(GetSessionRequest) returns (Session);
    rpc ListSessions(ListSessionsRequest) returns (stream Session);
    rpc SendInput(SendInputRequest) returns (google.protobuf.Empty);
    rpc ResizePty(ResizePtyRequest) returns (google.protobuf.Empty);
}

message CreateSessionRequest {
    string name = 1;
    ExecutionMode mode = 2;
    BackendType backend = 3;
    ResourceSpec resources = 4;
    string command = 5;
    string working_dir = 6;
    map<string, string> environment = 7;
}

message IOEvent {
    oneof event {
        OutputEvent output = 1;
        InputAck input_ack = 2;
        ResizeEvent resize = 3;
        SessionEvent session = 4;
    }
}

message OutputEvent {
    string pane_id = 1;
    bytes data = 2;
    google.protobuf.Timestamp timestamp = 3;
}

// === Scheduler Service ===
service SchedulerService {
    rpc Schedule(ScheduleRequest) returns (ScheduleResponse);
}

```

```

rpc CancelSchedule(CancelScheduleRequest) returns (google.protobuf.Empty);
rpc GetResourcePool(google.protobuf.Empty) returns (ResourcePool);
rpc Reserve(ReserveRequest) returns (Reservation);
rpc ReleaseReservation(ReleaseRequest) returns (google.protobuf.Empty);
rpc WatchPool(WatchPoolRequest) returns (stream PoolEvent);
}

message ScheduleRequest {
  string session_id = 1;
  int32 priority = 2;
  string requirements = 3; // ClassAds expression
  string rank = 4;         // ClassAds expression
  ResourceSpec resources = 5;
  ExecutionMode mode = 6;
}

// === Health Service ===
service HealthService {
  rpc GetClusterHealth(google.protobuf.Empty) returns (ClusterHealth);
  rpc GetNodeHealth(GetNodeHealthRequest) returns (HealthScore);
  rpc StreamHealth(stream HealthReport) returns (stream HealthAdvice);
  rpc PredictFailures(PredictRequest) returns (PredictResponse);
}

// === Advisory Service (LLM Brain) ===
service AdvisoryService {
  rpc ListAdvisories(ListAdvisoriesRequest) returns (stream Advisory);
  rpc ApproveAdvisory(ApproveRequest) returns (Advisory);
  rpc RejectAdvisory(RejectRequest) returns (Advisory);
  rpc GetExplanation(ExplanationRequest) returns (Explanation);
}

```

14. IMPLEMENTATION PHASES

Phase Overview

Phase	Name	Duration	Components	Deliverable
0	Foundation	4 weeks	Build system, CI/CD, core libraries	Running dev environment
1	Core Infras- tructure	8 weeks	Networking, discovery, messaging	Nodes can join/leave
2	Resource Management	6 weeks	Scheduler, GPU engine, monitoring	Jobs can be scheduled
3	Session Manager	8 weeks	Distributed tmux, I/O forwarding, migration	Interactive sessions work
4	Build Service	4 weeks	RBE protocol, distcc integration	AOSP builds distribute

Table 12 – continued

Phase	Name	Duration	Components	Deliverable
5	LLM Brain	6 weeks	Advisory system, verification, policies	Self-tuning active
6	Security Hardening	4 weeks	Zero Trust, audit, compliance	Production security
7	QA & Testing	6 weeks	HelixQA integration, chaos, perf	Production readiness
8	Polish & Release	4 weeks	Docs, setup wizard, packaging	v1.0 release
Total		50 weeks		

Detailed Phase Breakdown

PHASE 0: Foundation (Weeks 1-4) 0.1 Project Bootstrap

- 0.1.1 Create monorepo structure (Go modules, Zig build, C makefiles)
- 0.1.2 Set up CI/CD pipeline (GitHub Actions → ArgoCD)
- 0.1.3 Define coding standards (HelixConstitution enforcement)
- 0.1.4 Set up development environment (Docker Compose, Tilt)
- 0.1.5 Create integration testing framework (Testcontainers-Go)
- 0.1.6 Set up build system (Bazel or Make-based)

0.2 Core Libraries

- 0.2.1 Zig serialization library (Cap'n Proto compatible)
- 0.2.2 Go shared utilities (errors, logging, config)
- 0.2.3 WireGuard Go wrapper library
- 0.2.4 etcd client wrapper with caching
- 0.2.5 PostgreSQL connection pool with migrations
- 0.2.6 Redis client with cluster support
- 0.2.7 NATS client wrapper
- 0.2.8 gRPC service boilerplate
- 0.2.9 WebSocket library with reconnection
- 0.2.10 ClassAds parser and evaluator

0.3 Infrastructure

- 0.3.1 Docker Compose development stack
- 0.3.2 Local Kubernetes (kind/k3d) for testing
- 0.3.3 Prometheus + Grafana local setup
- 0.3.4 NATS + JetStream local setup

- 0.3.5 PostgreSQL + Redis local setup
- 0.3.6 etcd local cluster setup

PHASE 1: Core Infrastructure (Weeks 5-12) 1.1 Node Discovery Service

- 1.1.1 SWIM gossip protocol implementation
- 1.1.2 Node registration and deregistration
- 1.1.3 Heartbeat mechanism with phi accrual
- 1.1.4 mDNS local discovery
- 1.1.5 Bootstrap node rendezvous
- 1.1.6 etcd integration for persistent state
- 1.1.7 Node resource fingerprinting (CPU, memory, GPU)
- 1.1.8 Node capability advertisement
- 1.1.9 Split-brain detection and prevention
- 1.1.10 Network partition handling

1.2 WireGuard Mesh

- 1.2.1 WireGuard kernel integration
- 1.2.2 Automatic key generation and distribution
- 1.2.3 Mesh topology formation
- 1.2.4 NAT traversal (STUN/TURN/ICE)
- 1.2.5 SSH tunnel fallback
- 1.2.6 Connection health monitoring
- 1.2.7 Automatic reconnection
- 1.2.8 Multi-path routing (when available)

1.3 Messaging Infrastructure

- 1.3.1 NATS server deployment
- 1.3.2 JetStream configuration
- 1.3.3 Topic/channel definitions
- 1.3.4 Publisher/consumer patterns
- 1.3.5 Request/reply patterns
- 1.3.6 Event schema definitions (Avro)
- 1.3.7 Dead letter queue handling
- 1.3.8 Backpressure management

1.4 API Gateway

- 1.4.1 Gin Gonic HTTP server

- 1.4.2 gRPC-Gateway setup
- 1.4.3 Authentication middleware (mTLS)
- 1.4.4 Authorization middleware (OPA)
- 1.4.5 Rate limiting middleware
- 1.4.6 Request logging and tracing
- 1.4.7 WebSocket upgrade handling
- 1.4.8 Health check endpoint
- 1.4.9 Metrics exposition
- 1.4.10 API versioning strategy

PHASE 2: Resource Management (Weeks 13-18) 2.1 Resource Aggregator

- 2.1.1 Resource pool data model
- 2.1.2 Node resource collection (cgroups, /proc)
- 2.1.3 GPU resource detection (all vendors)
- 2.1.4 Resource update propagation
- 2.1.5 Utilization metrics calculation
- 2.1.6 Historical usage tracking
- 2.1.7 Capacity planning projections
- 2.1.8 Resource quota enforcement

2.2 Scheduler

- 2.2.1 Omega-model shared state implementation
- 2.2.2 etcd optimistic concurrency control
- 2.2.3 Scheduling pipeline (12 extension points)
- 2.2.4 NodeResourcesFit plugin
- 2.2.5 NodeAffinity plugin
- 2.2.6 TopologyAware plugin
- 2.2.7 CapabilityMatch plugin (ClassAds)
- 2.2.8 PrioritySort plugin
- 2.2.9 GangScheduling plugin
- 2.2.10 LoadAware plugin
- 2.2.11 Scheduling queue management
- 2.2.12 Preemption logic
- 2.2.13 Resource reservation system
- 2.2.14 Autoscaler integration hooks

2.3 GPU Compute Engine

- 2.3.1 GPU backend interface definition
- 2.3.2 NVIDIA CUDA backend
- 2.3.3 AMD ROCm backend
- 2.3.4 Intel oneAPI backend
- 2.3.5 Apple MLX backend
- 2.3.6 SYCL cross-platform backend
- 2.3.7 GPU device detection and registration
- 2.3.8 GPU memory management
- 2.3.9 GPU sharing (MPS, time-slicing)
- 2.3.10 GPU metrics collection
- 2.3.11 DRA-compatible API
- 2.3.12 HAMi-style interception (optional)

2.4 Health Monitor

- 2.4.1 Prometheus metrics collection
- 2.4.2 eBPF probe integration
- 2.4.3 Node health scoring algorithm
- 2.4.4 Component health checks
- 2.4.5 Failure prediction model (LSTM)
- 2.4.6 Anomaly detection (isolation forest)
- 2.4.7 Alert generation and routing
- 2.4.8 Self-healing action executor
- 2.4.9 Grafana dashboard provisioning
- 2.4.10 Historical trend analysis

PHASE 3: Session Manager (Weeks 19-26) 3.1 Session Backend Abstraction

- 3.1.1 SessionBackend interface definition
- 3.1.2 tmux backend implementation
- 3.1.3 Zellij backend implementation
- 3.1.4 screen backend implementation
- 3.1.5 Native backend (custom PTY)
- 3.1.6 Backend discovery and selection
- 3.1.7 Backend health monitoring

3.2 Distributed Session Core

- 3.2.1 Session lifecycle management
- 3.2.2 Window management
- 3.2.3 Pane management
- 3.2.4 Distributed pane placement
- 3.2.5 CRDT state synchronization
- 3.2.6 Session metadata persistence
- 3.2.7 Session routing table

3.3 I/O Forwarding

- 3.3.1 PTY master/slave setup
- 3.3.2 WebSocket I/O stream
- 3.3.3 ZeroMQ I/O proxy
- 3.3.4 Input handling (keyboard, mouse)
- 3.3.5 Output rendering (ANSI, UTF-8)
- 3.3.6 Terminal emulation (xterm-256color)
- 3.3.7 Resize handling
- 3.3.8 Copy-paste buffer
- 3.3.9 Unicode and wide character support

3.4 Session Migration

- 3.4.1 CRIU integration
- 3.4.2 DMTCP integration
- 3.4.3 Checkpoint creation
- 3.4.4 Checkpoint streaming (Arrow Flight)
- 3.4.5 Restore on target node
- 3.4.6 TCP connection repair/proxy
- 3.4.7 PTY state restoration
- 3.4.8 Seamless client handover
- 3.4.9 Migration failure handling
- 3.4.10 Automatic migration triggers

3.5 htmux CLI

- 3.5.1 CLI framework (cobra)
- 3.5.2 tmux-compatible commands
- 3.5.3 Session listing and selection
- 3.5.4 Interactive session attach

- 3.5.5 Configuration file (.htmux.conf)
- 3.5.6 Shell completions
- 3.5.7 Man pages
- 3.5.8 Color themes
- 3.5.9 Status bar customization

PHASE 4: Build Service (Weeks 27-30) 4.1 Bazel RBE Implementation

- 4.1.1 RBE protocol server
- 4.1.2 Action cache (content-addressed)
- 4.1.3 CAS (Content-Addressed Storage)
- 4.1.4 Execution service
- 4.1.5 Worker pool management
- 4.1.6 Buildbarn-compatible API

4.2 AOSP Integration

- 4.2.1 AOSP build detection
- 4.2.2 Soong/Blueprint analysis
- 4.2.3 distcc worker pool
- 4.2.4 ccache/sccache integration
- 4.2.5 Ninja job distribution
- 4.2.6 Build artifact caching
- 4.2.7 Incremental build support
- 4.2.8 Build progress reporting

4.3 Generic Batch Jobs

- 4.3.1 Batch job submission API
- 4.3.2 Checkpoint/restart for long jobs
- 4.3.3 Output streaming
- 4.3.4 Job dependency management
- 4.3.5 Priority and preemption
- 4.3.6 Result collection

PHASE 5: LLM Brain (Weeks 31-36) 5.1 LLMsVerifier Integration

- 5.1.1 Go SDK integration
- 5.1.2 Circuit breaker configuration
- 5.1.3 Model provider adapters (Kimi, DeepSeek, Claude)
- 5.1.4 Verification pipeline

- 5.1.5 Response validation rules
- 5.1.6 Fallback and retry logic

5.2 Advisory System

- 5.2.1 RAG knowledge base
- 5.2.2 Document ingestion (docs, runbooks)
- 5.2.3 Context window management
- 5.2.4 Chain-of-thought generation
- 5.2.5 Advisory creation pipeline
- 5.2.6 Risk assessment scoring
- 5.2.7 Auto-approval logic
- 5.2.8 Human review queue

5.3 Learning & Adaptation

- 5.3.1 Metrics ingestion pipeline
- 5.3.2 Pattern recognition
- 5.3.3 Reinforcement learning from feedback
- 5.3.4 Configuration optimization suggestions
- 5.3.5 Failure prediction model training
- 5.3.6 Anomaly detection tuning
- 5.3.7 Policy learning from approvals

5.4 Constitutional Enforcement

- 5.4.1 HelixConstitution parser
- 5.4.2 Rule engine integration
- 5.4.3 Safety constraint validation
- 5.4.4 Explanation generation
- 5.4.5 Override audit logging

PHASE 6: Security Hardening (Weeks 37-40) 6.1 Zero Trust Implementation

- 6.1.1 SPIFFE/SPIRE deployment
- 6.1.2 mTLS everywhere enforcement
- 6.1.3 OPA policy engine
- 6.1.4 RBAC implementation
- 6.1.5 Network policies (micro-segmentation)
- 6.1.6 Secrets management (Vault integration)

6.2 Audit & Compliance

- 6.2.1 Comprehensive audit logging
- 6.2.2 Immutable log storage
- 6.2.3 Compliance reporting
- 6.2.4 Intrusion detection (Falco)
- 6.2.5 Vulnerability scanning
- 6.2.6 Supply chain security (SBOM)

PHASE 7: QA & Testing (Weeks 41-46) 7.1 Test Infrastructure

- 7.1.1 HelixQA integration
- 7.1.2 Test suite automation
- 7.1.3 Mutation testing
- 7.1.4 Property-based testing
- 7.1.5 Integration test matrix (all hardware)

7.2 Chaos Engineering

- 7.2.1 Chaos mesh deployment
- 7.2.2 Node failure scenarios
- 7.2.3 Network partition scenarios
- 7.2.4 Resource exhaustion scenarios
- 7.2.5 Automatic recovery validation

7.3 Performance Testing

- 7.3.1 Load testing (k6)
- 7.3.2 Benchmark suite
- 7.3.3 Regression detection
- 7.3.4 Scalability testing (up to 64 nodes)
- 7.3.5 GPU throughput testing

7.4 Formal Verification

- 7.4.1 TLA+ specification for consensus
- 7.4.2 TLA+ specification for scheduling
- 7.4.3 Model checking
- 7.4.4 Safety property proofs

PHASE 8: Polish & Release (Weeks 47-50) 8.1 Setup Wizard

- 8.1.1 Single-command install script
- 8.1.2 Hardware auto-detection
- 8.1.3 Automatic driver installation

- 8.1.4 Mesh network auto-formation
- 8.1.5 First-run configuration
- 8.1.6 Progress reporting
- 8.1.7 Error recovery

8.2 Documentation

- 8.2.1 Architecture documentation
- 8.2.2 API documentation (generated)
- 8.2.3 User guide
- 8.2.4 Administrator guide
- 8.2.5 Developer guide
- 8.2.6 Troubleshooting guide

8.3 Packaging

- 8.3.1 Debian/Ubuntu packages
- 8.3.2 macOS packages (Homebrew)
- 8.3.3 Docker images
- 8.3.4 Helm charts
- 8.3.5 Release automation

15. RISK ANALYSIS & MITIGATION

Critical Risks

#	Risk	Probability	Impact	Mitigation
R1	Apple Silicon compatibility issues	High	High	Early prototyping with M3 Pro; separate MLX backend; Rosetta 2 fallback
R2	Performance degradation over Gigabit Ethernet	Medium	High	Zero-copy data paths (Arrow Flight); compression; local caching; upgrade path to 10GbE
R3	Session migration failure (CRIU limitations)	Medium	High	DMTCP fallback; graceful restart option; transparent reconnection
R4	GPU backend fragmentation (4 vendors)	High	Medium	SYCL abstraction; DRA compatibility; community contribution model

Table 13 – continued

#	Risk	Probability	Impact	Mitigation
R5	LLM hallucination causing bad decisions	Medium	Critical	LLMsVerifier mandatory; advisory-only mode; human approval for risky changes
R6	Split-brain in network partitions	Low	Critical	Raft consensus; quorum requirements; automatic fencing
R7	Security vulnerability in mesh VPN	Low	Critical	WireGuard audit; mTLS layered; rapid patch deployment
R8	etcd performance at scale (>100 nodes)	Medium	Medium	MultiRaft; sharding; eventual consistency for non-critical state
R9	Build service incompatibility with AOSP	Medium	High	Bazel RBE standard protocol; multiple backend support; community testing
R10	Project scope creep	High	High	Phased delivery; MVP first; strict prioritization; automated scope gates

Contingency Plans

If CRIU proves unreliable: Fall back to DMTCP for Linux, graceful session restart for macOS. Market as “session persistence” rather than “live migration” initially.

If GPU abstraction fails: Ship with NVIDIA-only support first. Add AMD/Intel/Apple in subsequent releases. Use cloud GPU as fallback.

If performance is unacceptable: Document minimum requirements (2x Gigabit NICs, NVMe SSD). Provide performance tuning guide. Consider DPDK for data plane.

If LLM Brain is too risky: Ship as optional plugin, disabled by default. Focus on rule-based optimization initially. Enable LLM features after proven safe.